

C S C 2 4 1

Python Programming Language I

Study Manual

Victor Ayodeji

```
>>> total = 0
>>> for n in scores:
    total += n
>>> print(total / len(scores))

72.5 ■
```

Contents

Every line is a link. The page numbers are read back from the rendered book, never typed by hand.

How to use this manual	2
MODULE ONE · INTRODUCTION TO PYTHON PROGRAMMING	4
Unit 1 · Overview of Python	5
Unit 2 · Setting Up Python	8
MODULE TWO · PYTHON BASICS AND SYNTAX	12
Unit 1 · Python Language Syntax	13
Unit 2 · Variables and Data Types	19
Unit 3 · Python Operators	24
Unit 4 · Working with Strings	29
MODULE THREE · CONTROL FLOW AND DATA STRUCTURES	39
Unit 1 · Decision Making in Python	40
Unit 2 · Loops: Repetition and Branching	44
Unit 3 · Lists and Tuples	50
Unit 4 · Sets and Dictionaries	55
MODULE FOUR · FUNCTIONS, MODULES AND FILE HANDLING	60
Unit 1 · Functions in Python	61
Unit 2 · Modules and Packages	66
Unit 3 · File Handling in Python	67
Unit 4 · Exception Handling	72
MODULE FIVE · DATABASES AND GUI DEVELOPMENT	76
Unit 1 · Python and Databases	77
Unit 2 · Python GUI Programming	85
MOCK EXAMINATION ONE	89
Question Paper	90
Answers	98
MOCK EXAMINATION TWO	116
Question Paper	117
Answers	125

How to use this manual

Read this page once. It is short, and it decides whether the rest of the manual works for you.

TEACH

THE BARGAIN

This manual assumes you have never written a line of code, and it carries you from there to a full mark in the examination. It is also meant to be **enough on its own**. You should never need a lecturer, a website or the official documentation to follow it. Anything it uses, it teaches first.

In exchange it asks one thing: **type the code**. Reading code creates a feeling of understanding that disappears in the examination hall. Typing it, getting it wrong, and fixing it is the thing that actually teaches. Every code block here was run on a real interpreter before it was printed, so if yours does something different, the difference is information.

The boxes

Every page is built from a small set of boxes. Each has one job, and the colour tells you which before you read a word.

TEACH

New ideas, explained from nothing. Read these properly.

● MUST MEMORISE

Python yellow means **learn this by heart**. It is the only thing this colour is ever used for in this manual. Nothing decorative is yellow. If a page has yellow on it, there is something on that page you are expected to know cold.

WORKED

A problem solved in numbered steps, then a **now you try** at the end. Do the "now you try" before moving on.

DRY RUN

A program traced by hand, one line at a time, with a table of what every variable holds. This examination asks you to state what code displays, and this is how you work it out without a computer. **Dry run** is the term used in this course.

⚠ TRAP

A mistake that is easy to make and costly. Most of these are taken from the errors the examination itself plants in its code.

RECALL

Questions with the answers printed underneath, below a dashed line. **Cover the answers.** Say your answer out loud, or write it, and only then look. Checking whether you knew something is not the same as knowing it, and the difference is the whole point of this box.

Reading the code

● MUST MEMORISE

THREE KINDS OF BLOCK

Code appears in three kinds of block, and confusing them is the most common early mistake. Each has a different coloured left edge.

1. The interactive prompt. Lines starting `>>>` are what you type. Lines without it are what Python printed back. You never type the `>>>` itself.

```
>>> 2 + 2
4
```

2. A script. The contents of a file, to be saved and run as a whole. No `>>>` anywhere.

```
print("saved in a file")
```

3. Output. What appeared on screen. You never type this.

```
saved in a file
```

Where a keystroke is meant rather than something to type, it is shown as a key: press

Enter .

TEACH

ONE PROMISE ABOUT OUTPUT

Every output shown in this manual was captured from a real Python 3.13 interpreter at the moment this page was built. None was typed by hand from memory. This matters more than it sounds: a printed output that was never actually run is just a confident guess, and where Python's real answer is surprising, a guess is exactly where it will be wrong. You will meet one such surprise in Module Two, Unit 2, and it is the reason for this promise.

MODULE ONE

Introduction to Python Programming

What a program actually is, what the machine does with it, and how to get Python running on your own computer so that every page after this one is something you can try rather than something you read.

```
>>> print( "hello" )
```

```
hello
```

Overview of Python

Start here even if you have never written a line of code. This unit assumes nothing.

By the end of this unit you can: say what a program is, explain what an interpreter does and how it differs from a compiler, state why Python is used, and name the places Python is used in practice.

What a program is

TEACH

FROM ZERO

A computer does nothing on its own. It waits to be told, in exact terms, what to do. A **program** is that telling: an ordered list of instructions, written down, that the computer carries out one after another.

Think of a recipe. A recipe is not the cooking; it is the written instructions that produce the cooking when somebody follows them. A program is not the work; it is the written instructions that produce the work when the computer follows them.

Two words that follow from this, and that you will use constantly:

- ▶ **Code** is the text you write. The instructions themselves.
- ▶ **Running** (or **executing**) a program means telling the computer to carry out those instructions now, from the top, in order.

Why we need a language

TEACH

The computer's own vocabulary is numbers. Nothing else. Instructions inside the machine are patterns of the digits 0 and 1, called **machine code**. Writing anything useful directly in 0s and 1s is possible and unbearable.

So we write in a **programming language** instead: a fixed, small, precisely defined set of words and symbols that a human can read and that can be turned into machine code automatically. Python is one such language.

A programming language is deliberately far stricter than English. In English, "print hello" and "hello, print it" and "could you print hello" all land. A programming language accepts exactly one form and rejects the rest. That strictness is not the language being difficult. It is the reason the translation to machine code can be done without a human guessing at your intention.

Interpreter and compiler

Something must sit between your text and the machine's numbers. There are two ways to build that something, and the difference is examinable.

A **compiler** reads your *whole* program, translates all of it into machine code, and writes that machine code out as a new file. Nothing runs during translation. You run the produced file afterwards, and it runs on its own.

An **interpreter** reads your program *one instruction at a time*, and carries out each instruction as soon as it has read it. Nothing is written out. There is no separate file to run afterwards. Translation and running are the same activity.

Python is interpreted. The program that reads your Python and does what it says is called **the Python interpreter**, and installing Python (Unit 2) means installing it.

● MUST MEMORISE

COMPILER VS INTERPRETER

	COMPILER	INTERPRETER
How much it reads before acting	The whole program	One instruction
Produces a machine code file?	Yes	No
When errors surface	All at translation, before any run	When that line is reached
Speed of the finished program	Faster	Slower
Example language	C, C++	Python

Marks are lost here by naming only one difference. The question asks you to differentiate, so give the pair each time: what it reads, and what it produces.

⚠ TRAP

"Interpreted means Python has no errors until it runs" is **half true and the half that is false costs marks**. Python still checks that each line is *grammatically* possible before running anything in that file. A missing colon is caught immediately, before the first line executes. What waits until the line is reached is a *meaning* error, such as dividing by zero.

You will meet this split again as `SyntaxError` versus runtime error in Module Two, and it is worth marks on its own.

Why Python

TEACH

- ▶ **It reads close to English.** Fewer symbols stand between your intention and the page, so there is less to get wrong.
- ▶ **It is short.** Work that takes many lines elsewhere often takes one here.
- ▶ **It comes with a large standard library:** a big collection of ready made code that ships with Python, so common jobs are already solved. You will use one of these, `sqlite3`, in Module Five.
- ▶ **It is portable.** The same Python file runs on Windows, macOS and Linux.
- ▶ **It is free** and its interpreter is open source.

TEACH

WHERE PYTHON IS USED

FIELD	WHAT PYTHON DOES THERE
Web development	Runs the server behind a website
Data analysis	Reads large data files, summarises and charts them
Artificial intelligence	Builds and trains models
Automation and scripting	Repetitive machine work, such as renaming thousands of files
Scientific computing	Simulation and numerical work
Software testing	Automatically exercising other programs

RECALL

COVER THE ANSWERS

1. What is a program?
2. Give the two differences between a compiler and an interpreter that earn the marks.
3. Is Python compiled or interpreted?
4. Name three fields where Python is used.

ANSWERS

1. An ordered list of instructions that a computer carries out, written down in a language it can be made to understand.
2. A compiler reads the whole program and produces a machine code file. An interpreter reads one instruction at a time and carries it out at once, producing no file.
3. Interpreted.
4. Any three of: web development, data analysis, artificial intelligence, automation, scientific computing, software testing.

Setting Up Python

Do this once, on your own machine, before you read Module Two. Everything after here is meant to be typed, not watched.

By the end of this unit you can: install Python, confirm it installed, use the interactive prompt, write and run a script file, and say which of the two you are looking at.

Installing

TEACH

WINDOWS

1. Open `python.org` and choose Downloads. The site offers you the installer for the system you are on.
2. Run the installer.
3. **Tick "Add Python to PATH" on the first screen.** This is the one step that people skip and then cannot proceed. It is explained directly below.
4. Choose "Install Now" and wait for it to finish.

● MUST MEMORISE

PATH

PATH is a list your operating system keeps of the folders it will search when you type a command. If Python's folder is not on that list, then typing `python` gets you "command not found", even though Python is installed perfectly well. The program is there; the system just does not know where to look.

If you have already installed without ticking it, run the installer again, choose Modify, and add it. You do not need to uninstall anything.

TEACH

MACOS AND LINUX

Both usually ship with a Python already, but often an old one. Install the current version from `python.org` in the same way. On Linux you may instead use your system's package manager, for example `sudo apt install python3` on Ubuntu.

On macOS and Linux the command is often `python3` rather than `python`. If `python` gives you an error or a version starting with 2, use `python3` everywhere in this manual.

Confirming it worked

TEACH

Open a **terminal**, the window where you type commands to your operating system rather than clicking. On Windows it is called Command Prompt or PowerShell; on macOS and Linux it is called Terminal. Type this and press **Enter**:

IN THE TERMINAL, NOT IN PYTHON

```
python --version
```

It should answer with a version number:

```
Python 3.13.5
```

Your exact numbers will differ, and that is fine. Any version starting with **3** is fine. If it starts with 2, or you get "command not found", see the PATH box above, and try `python3 --version`.

⚠ TRAP

PYTHON 2

If your version starts with **2**, stop and fix it. Python 2 is a different language in small, silent ways. In Python 2, `5 / 2` is `2`. In Python 3 it is `2.5`. Nothing warns you; you simply get wrong answers. Every page of this manual is Python 3.

The two ways to run Python

This is the distinction that confuses beginners more than any other, so it gets its own box. From here on, every code block in this manual is labelled as one or the other.

● MUST MEMORISE

PROMPT VS SCRIPT

1. The interactive prompt (also called the REPL, or interactive mode). You type one line, press **Enter**, and Python answers immediately. You recognise it by the three symbols `>>>` at the left, which Python prints to mean "your turn". It forgets everything when you close it.

2. A script. You type many lines into a file, save it with a name ending in `.py`, and run the whole file at once. It keeps everything, because it is a file.

The prompt is for asking. A script is for building. Your exam answers are scripts.

Type `python` on its own in the terminal and press `Enter`. The prompt appears:

```
>>>
```

Now type an instruction and press `Enter`. Python answers on the next line, then offers you the prompt again:

```
>>> print("Hello, World!")
Hello, World!
>>>
```

To leave the prompt and return to the terminal, type `exit()` and press `Enter`.

● MUST MEMORISE

READING TRANSCRIPTS

In every prompt transcript in this manual:

- ▶ Lines beginning `>>>` are what **you type**. Do not type the `>>>` itself; Python prints that.
- ▶ Lines with no `>>>` are what **Python printed back**.

So in the transcript above, you typed 21 characters and Python printed `Hello, World!`. Getting this backwards is the single most common early confusion, which is why the two are given different coloured edges throughout.

Open any plain text editor. Notepad works. So does IDLE, which was installed alongside Python: find it in your programs, then choose File, then New File.

Type these two lines:

HELLO.PY

```
print("Hello, World!")
print("I am running from a file.")
```

Save it as `hello.py`. The `.py` ending is what marks a file as Python. Then, in the terminal, move to the folder you saved it in and type:

IN THE TERMINAL

```
python hello.py
```

Python reads the file from the top, carries out each line in order, prints:

```
Hello, World!
I am running from a file.
```

and then exits. Notice there is no `>>>` anywhere: you are not having a conversation, you are handing over a finished list of instructions.

Typing `python hello.py` while already at the `>>>` prompt gives you a `SyntaxError`. Those are terminal commands, not Python instructions. They live in different places:

```
>>> python hello.py
File "<stdin>", line 1
  python hello.py
    ^^^^^
SyntaxError: invalid syntax
```

If you see this, you are at the Python prompt when you meant to be at the terminal. Type `exit()` first.

RECALL

COVER THE ANSWERS

1. What does PATH do, and what goes wrong if Python is not on it?
2. What do the three symbols `>>>` tell you?
3. Which ending must a Python file's name have?
4. You want to keep your work and hand it in. Prompt or script?

ANSWERS

1. PATH lists the folders the operating system searches when you type a command. If Python's folder is missing from it, typing `python` reports "command not found" even though Python is installed.
2. That you are at the interactive prompt, and that Python is waiting for you to type one instruction. You never type the symbols yourself.
3. `.py`
4. A script. The prompt forgets everything when it closes.

MODULE TWO

Python Basics and Syntax

The grammar of the language, the boxes you keep values in, and the symbols that combine them. Everything in the rest of the manual is built out of what is in these three units.

```
age = 25
```

```
name = "Ada" ■
```

Python Language Syntax

Syntax means the grammar: the rules that decide whether a line is Python at all, before anyone asks whether it is correct.

By the end of this unit you can: write and run statements, use comments, explain why indentation is not decoration in Python, print with `sep` and `end`, use escape characters, format with f-strings, and read a traceback.

Statements

TEACH

A **statement** is one complete instruction. In Python, one statement occupies one line, and the end of the line ends the statement. There is no symbol to mark the end.

```
print("first")
print("second")
```

Two lines, two statements, carried out top to bottom. If you know a language that ends statements with a semicolon, note that Python does not want one. It is not an error, but it marks your code as written by somebody thinking in another language, and the marks in this course go to Python style.

Comments

TEACH

A **comment** is a note to a human. Python ignores it completely. Everything from a `#` to the end of that line is a comment.

```
# Work out the area of the room.
area = length * width      # in square metres
```

Comments do not run, do not print, and do not affect anything. They exist so that you, in three weeks, can tell what you meant.

● MUST MEMORISE

READ THIS ONCE, CAREFULLY

A comment is **never** output. If you see

```
>>> 10 % 2
0      # Even
```

then the output is `0`. The words `# Even` are a human annotation sitting beside it, not something Python said. Python never answers you in comments.

This matters because printed material sometimes puts the annotation where the output should be, and a reader new to the language cannot tell which is which. In this manual the rule is absolute: **output is shown on its own line, in an output block, exactly as a real interpreter printed it.** Every output in this manual was captured from a run, not typed by hand.

Indentation

TEACH

PYTHON'S ONE BIG IDEA

In most languages, spaces at the start of a line are decoration, and blocks of code are marked with braces. **In Python the spaces are the structure.** There are no braces. Indentation is how Python knows which lines belong to what.

You have not met `if` yet, and it arrives properly in Module Three. Read this only for the shape:

```
if raining:
    print("take a coat")      # indented: belongs to the if
print("leave the house")    # not indented: always runs
```

The indented line runs only when it is raining. The unindented line always runs. Move that one line four spaces to the left or right and you have changed what the program does. Nothing else changed.

The rule: a colon at the end of a line means "a block follows", and that block is the lines indented beneath it. Use **four spaces** per level. Be consistent: Python accepts any consistent width, and rejects a mixture.

⚠ TRAP

EXAM FAVOURITE

Indentation errors are a standing question in this course. Two distinct failures, and the wording of the error tells you which:

```
IndentationError: expected an indented block after 'if' statement on line 1
```

means you wrote the colon and then did not indent anything beneath it.

```
IndentationError: unexpected indent
```

means you indented a line that had no colon above it to belong to.

Printing

TEACH

`print()` displays things. Give it several items separated by commas and it prints them all, putting **one space** between each:

```
>>> print("Ada", 25, "Lagos")
Ada 25 Lagos
```

It then moves to a new line. Both of those behaviours can be changed, and each has a name you must know.

`sep` sets what goes **between** the items. Default: one space.

`end` sets what goes **after** the last item. Default: a newline, written `\n`.

```
>>> print("2026", "07", "17", sep="-")
2026-07-17
>>> print("Ada", "Bob", sep="")
AdaBob
```

Setting `end` to the empty string stops the newline, so the next `print` continues on the same line:

```
>>> print("Hello", end="")
>>> print("World")
HelloWorld
```

Read that transcript carefully. The first `print` displayed `Hello` and stayed put; the second added `World` and ended the line. The output `HelloWorld` is the work of both statements together.

Quotes and escape characters

TEACH

Text in Python is written inside quotes, and is called a **string**. Single and double quotes are equally valid and mean exactly the same thing:

```
'Ada'
"Ada"
```

Having both is useful, because a string in double quotes may freely contain a single quote:

```
>>> print("It's fine")
It's fine
```

The problem arrives when the string must contain the same quote that ends it. Python reads the second quote as the end of the string and the rest becomes nonsense. The fix is an **escape character**: a backslash `\` before the quote, which tells Python "this next character is data, not punctuation".

```
>>> print("She said, \"Python is cool!\")
She said, "Python is cool!"
```


ESCAPE	MEANS
<code>\n</code>	Newline: move to the next line
<code>\t</code>	Tab: jump to the next tab stop
<code>\"</code>	A double quote, as data
<code>\'</code>	A single quote, as data
<code>\\</code>	One backslash, as data

A question asking you to "list three escape characters and their effects" has appeared in this course. The three safest to give are `\n`, `\t` and `\\`.

TEACH

ONE STRING, TWO LINES

`\n` inside a string breaks it across lines when printed. It is one string, not two:

```
>>> print("Name: Ada\nCity: Lagos")
Name: Ada
City: Lagos
```

f-strings

TEACH

You will constantly want to put a value inside a sentence. The direct way is an **f-string**: put the letter `f` immediately before the opening quote, then write `{ }` wherever a value belongs.

```
>>> name = "Ada"
>>> age = 25
>>> print(f"{name} is {age}")
Ada is 25
```

The `f` is not optional and not decoration. Without it you get the braces themselves:

```
>>> print("{name} is {age}")
{name} is {age}
```

After a colon inside the braces you may say *how* to display the value. These are the ones this course uses:

WRITTEN	MEANS	EXAMPLE IN	EXAMPLE OUT
<code>:.2f</code>	Fixed to 2 decimal places	123.456	123.46
<code>,</code>	Comma between thousands	1250000	1,250,000
<code>:,.2f</code>	Both together	9876543.21	9,876,543.21
<code>:.2%</code>	As a percentage, 2 decimals	0.875	87.50%
<code>:<10</code>	Left aligned, width 10	'Item'	Item.....
<code>:>10</code>	Right aligned, width 10	'Price'	Price

In the last two rows each dot stands for one space, so that you can count them. Python prints spaces, not dots.

WORKED

ALIGNMENT YOU CAN ACTUALLY SEE

Field widths exist to build columns, and columns only work in a font where every character takes the same width. Every code block in this manual is set in such a font. Give three rows **the same format string** and they line up:

```
>>> print(f{'Item':<10}{'Price':>10})
Item          Price
>>> print(f{'Apple':<10}{1.5:>10.2f})
Apple         1.50
>>> print(f{'Banana':<10}{12.75:>10.2f})
Banana        12.75
```

The words start in the same column because `:<10` pins them to the left. The prices **end** in the same column because `:>10` pins them to the right, and that is true even though 1.50 and 12.75 have different lengths. That is what a field width buys you.

Spaces cannot be counted by eye, so here are those same three lines with every space drawn as a dot. Each dot is one space, and nothing else has changed:

```
Item.....Price
Apple.....1.50
Banana.....12.75
```

Count the first line. `Item` takes 4 of its 10 columns, so **6** spaces pad it on the right. `Price` is 5 characters pushed to the right hand edge of its own 10 columns, so **5** spaces come first. That is $6 + 5 = 11$ dots, which is what you counted. Every line is 20 characters wide.

Now you try: work out the dots for a row holding `'Fig'` and `3.5` in the same format, before you run it. (3 + 7 pad, then 6 + '3.50': Fig then 13 dots then 3.50.)

Reading a traceback

TEACH

WORTH MANY MARKS

When Python cannot do what you asked, it prints a **traceback**. Beginners see a wall of red and look away. It is the most useful text on the screen, and half of this course's code marks come from finding errors, so learn to read it.

```
Traceback (most recent call last):
  File "C:\Users\you\hello.py", line 3, in <module>
    print(10 / 0)
    ~~~^~~
ZeroDivisionError: division by zero
```

Read it **from the bottom up**:

- ▶ **Last line** is what went wrong: `ZeroDivisionError`, and in plain words, division by zero. This is the line that tells you the problem.
- ▶ **Above it** is the offending code, with the marks `~~~^~~` pointing at the exact part that failed. The `^` sits under `/`, the operation that could not be done.
- ▶ **Above that** is where: the full path of the file, and line 3.

Error type, then message, then place. Three facts, always in that order.

Python prints the file's whole location, so your first line will be longer than the one above and will show wherever you saved it. Only the file name and the line number matter to you.

● MUST MEMORISE

SYNTAX BEFORE RUNTIME

Errors come in two kinds and they happen at **different times**:

A `SyntaxError` is a grammar failure. Python finds it while reading the file, *before running a single line*. Nothing in the file executes. There is no traceback of the kind above, because nothing ran.

A runtime error is a meaning failure in a line that is grammatically fine. Python only finds it when it reaches that line, so everything before it has already run and printed.

The consequence, which is examinable: a `SyntaxError` anywhere in a file hides every other error in it. If a file has a missing bracket on line 2 and a misspelled name on line 9, you are told about line 2 only. Fix it, run again, and only then hear about line 9.

1. What does a colon at the end of a line announce?
2. What are `sep` and `end`?
3. Which end of a traceback do you read first?
4. A file has a `SyntaxError` on line 2 and a `NameError` on line 9. What does Python report?

ANSWERS

1. That a block follows, made of the lines indented beneath it.
2. `sep` is what `print` puts between items (default one space); `end` is what it puts after the last one (default a newline).
3. The bottom. The last line names the error and states it in words.
4. The `SyntaxError` on line 2, and nothing else. The file never runs, so line 9 is never reached.

MODULE TWO · UNIT 2

Variables and Data Types

Where values are kept, what kinds of value exist, and the one thing about numbers that surprises everybody exactly once.

By the end of this unit you can: create and reassign variables, name them legally, list the four basic types, check a type, read input, convert between types, and explain why `0.1 + 0.2` is not `0.3`.

Variables

TEACH

A **variable** is a name that refers to a value. You make one by writing the name, then `=`, then the value:

```
>>> age = 25
>>> age
25
```

The `=` here does **not** mean equals. It means *assign*: work out the value on the right, then make the name on the left refer to it. It is an instruction, not a statement of fact. This is why

```
count = count + 1
```

is perfectly sensible, though as mathematics it would be nonsense. It reads: take what `count` is now, add 1, and make `count` refer to that.

A variable can be given a new value at any time, including a value of a different kind. The old value is simply forgotten:

```
>>> x = 99
>>> print(x)
99
>>> x = "Take me to your leader."
>>> print(x)
Take me to your leader.
```

Python does not object. The name is a label, not a box of a fixed size.

● MUST MEMORISE

NAMING RULES

Must:

- ▶ Start with a letter or an underscore. **Never** a digit.
- ▶ Contain only letters, digits and underscores. No spaces, no hyphens.
- ▶ Not be a Python keyword (`if` , `for` , `class` , and the rest).

Should: be lowercase with underscores between words (`total_score`), and say what it holds. A name like `x` tells the reader nothing.

Case matters. `age` , `Age` and `AGE` are three different variables. This is a standing source of exam errors: a program that defines `square` and calls `Square` is broken.

The four basic types

● MUST MEMORISE

TYPES

TYPE	NAME IN PYTHON	WHAT IT HOLDS	EXAMPLES
Integer	<code>int</code>	Whole numbers, no decimal point	25, 0, -7
Float	<code>float</code>	Numbers with a decimal point	3.14, -0.5, 2.0
String	<code>str</code>	Text, in quotes	"Ada", 'x', ""
Boolean	<code>bool</code>	Truth, one of exactly two values	True, False

`True` and `False` are capitalised and are not strings. `"True"` in quotes is a string.

⚠ TRAP

2.0 IS NOT AN INT

`2` and `2.0` are equal in value but are different types. The decimal point makes it a float, even when there is nothing after it. This decides what division gives you back, as Unit 3 shows.

`type()` tells you what something is. When in doubt, ask:

```
>>> type(25)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type("Ada")
<class 'str'>
>>> type(True)
<class 'bool'>
```

Read `<class 'int'>` as "this is an int". The word class is Python's internal vocabulary for a type.

Input

● MUST MEMORISE

INPUT ALWAYS GIVES A STRING

`input()` stops the program, waits for the user to type something and press **Enter**, and hands back what they typed. The text you pass to it is shown as the prompt.

What it hands back is always a string. Always. Even when the user types digits.

```
age = input("How old are you? ")
print(type(age))
```

```
How old are you? 25
<class 'str'>
```

The user typed 25 and Python is holding the two characters "25", not the number. This single fact causes more broken exam answers than anything else in the course.

⚠ TRAP

THE CLASSIC

Forget to convert, and arithmetic does something silently absurd rather than failing:

```
a = input("First number: ") # user types 5
b = input("Second number: ") # user types 3
print(a + b)
```

```
First number: 5
Second number: 3
53
```

53, not 8. Because `+` on two strings joins them end to end. The program did exactly what you asked and nothing reported an error. The fix is to convert.

Converting between types

● MUST MEMORISE

CASTING

FUNCTION	URNS A VALUE INTO	EXAMPLE	RESULT
<code>int()</code>	an integer	<code>int("25")</code>	25
<code>float()</code>	a float	<code>float("3.5")</code>	3.5
<code>str()</code>	a string	<code>str(100)</code>	'100'

So the fix for the trap above, and the pattern you will write hundreds of times:

```
age = int(input("How old are you? "))
```

Read it from the inside out: `input` collects text, `int` turns that text into a number, `=` stores the number.

TEACH

INT() TRUNCATES

`int()` on a float throws the decimal part away. It does not round:

```
>>> int(3.9)
3
>>> int(-3.9)
-3
```

It cuts towards zero. Use `round()` if you want rounding.

⚠ TRAP

INT() IS FUSSY ABOUT STRINGS

`int()` can only convert a string that is entirely a whole number. Give it anything else and you get a runtime error, and the program stops:

```
>>> int("xyz")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'xyz'
```

Note that `int("3.5")` fails too, for the same reason: 3.5 is not a whole number written out. Use `float("3.5")`, or `int(float("3.5"))` if you truly want 3.

Floats are not exact

● MUST MEMORISE

EVERYBODY MEETS THIS ONCE

Type this at the prompt. It is not a trick:

```
>>> 0.1 + 0.2
0.30000000000000004
```

Python is not broken and you did not make a mistake. A computer stores numbers in binary, in a fixed amount of space. Some numbers that are short and exact in decimal, such as 0.1, cannot be written exactly in binary at all, in the same way that 1/3 cannot be written exactly in decimal: 0.333... never finishes. Python stores the closest value it can fit, and the tiny difference shows up when you add.

This is not a Python quirk. It is how nearly every language on nearly every machine stores decimals.

WORKED

WHERE IT WILL BITE YOU

It hides until it does not. Watch an ordinary looking interest calculation:

```
>>> INTEREST_RATE = 0.069
>>> balance = 10000
>>> amount = balance * INTEREST_RATE
>>> amount
690.00000000000001
```

Not 690.0 . And yet change the rate to 0.072 and it comes out perfectly clean:

```
>>> INTEREST_RATE = 0.072
>>> amount = balance * INTEREST_RATE
>>> amount
720.0
```

Same code, same shape, one is ugly and one is clean. That is exactly what makes this dangerous: you cannot predict which by looking, so you must handle it always rather than when it looks likely.

The rule: never show a raw float to a user, and never compare two floats with `==` . Format it on the way out.

● MUST MEMORISE

THE FIX

Format the float when you display it, and the problem disappears at the only point where it mattered:

```
>>> amount = 10000 * 0.069
>>> print(f"Interest: {amount:.2f}")
Interest: 690.00
```

Any exam program that reports an average, a percentage or an amount of money should print it through `:.2f` . It is one keystroke's worth of effort and it is the difference between a professional answer and a puzzling one.

1. What type does `input()` return?
2. The user types 5 and 3 into two `input()` calls and you print `a + b`. What appears, and why?
3. What is `int(3.9)`?
4. Why is `0.1 + 0.2` not `0.3`?

ANSWERS

1. A string, always, even if the user typed digits.
2. 53. Both are strings, and `+` on strings joins them. No error is reported.
3. 3. `int()` truncates towards zero, it does not round.
4. Floats are stored in binary in fixed space. 0.1 has no exact binary form, so Python stores the nearest value it can, and the small error becomes visible when adding. Format with `:.2f` on output.

MODULE TWO · UNIT 3

Python Operators

The symbols that combine values. Two of them, `//` and `%`, carry more exam marks than the rest put together.

By the end of this unit you can: use every arithmetic operator, predict what division returns, use `%` to test divisibility, apply precedence correctly, and use comparison and logical operators.

Arithmetic

● MUST MEMORISE

ALL SEVEN

OPERATOR	NAME	EXAMPLE	RESULT
+	Add	<code>7 + 5</code>	12
-	Subtract	<code>7 - 5</code>	2
*	Multiply	<code>7 * 5</code>	35
/	Divide	<code>7 / 5</code>	1.4
//	Floor divide	<code>7 // 5</code>	1
%	Remainder (modulus)	<code>7 % 5</code>	2
**	Exponent (power)	<code>7 ** 2</code>	49

● MUST MEMORISE

/ ALWAYS GIVES A FLOAT

/ gives a **float every time**, even when the division is exact:

```
>>> 10 / 2
5.0
>>> 5 / 2
2.5
```

10 / 2 is 5.0, not 5. If you need a whole number, use // or wrap it in int().

// divides and throws away the fractional part, giving the whole number of times the divisor fits:

```
>>> 7 // 5
1
>>> 17 // 3
5
```

● MUST MEMORISE

% IS THE WORKHORSE

% gives the **remainder** after division. 17 % 3 is 2, because 3 fits into 17 five times with 2 left over.

// and % are the two halves of one division:

```
>>> 17 // 3
5
>>> 17 % 3
2
```

17 divided by 3 is **5 remainder 2**. The first gives the 5, the second the 2.

The use that appears in this exam every year: a number is divisible by n exactly when `number % n == 0`.

```
>>> 10 % 2
0
>>> 7 % 2
1
```

A remainder of 0 on division by 2 means even. A remainder of 1 means odd. Read those two transcripts again: the outputs are 0 and 1. That is what Python printed.

This is `//` and `%` working together, and it is the standard demonstration of both. Turn 11730 seconds into hours, minutes and seconds.

- 1 An hour is 3600 seconds. **How many whole hours fit?** That is floor division.
11730 // 3600 gives 3
- 2 **What is left over after those hours?** Take the remainder on 3600, then ask how many whole minutes fit in it. Both steps at once: take the remainder on 60 of the total whole minutes.
(11730 // 60) % 60 gives 15
- 3 **Seconds left over** is the remainder on 60.
11730 % 60 gives 30

TIME_CONVERTER.PY

```
total_seconds = 11730
hours = total_seconds // 3600
minutes = (total_seconds // 60) % 60
seconds = total_seconds % 60
print("Hours:", hours)
print("Minutes:", minutes)
print("Seconds:", seconds)
```

```
Hours: 3
Minutes: 15
Seconds: 30
```

Now you try: change 11730 to 7325 and predict the three numbers before running it. (2 hours, 2 minutes, 5 seconds.)

● MUST MEMORISE

**** IS RIGHT ASSOCIATIVE**

Nearly every operator groups from the left. `**` groups from the **right**:

```
>>> 2 ** 3 ** 2
512
```

This is `2 ** (3 ** 2)`, which is `2 ** 9`, which is 512. It is **not** `(2 ** 3) ** 2`, which would be 64.

`**` also does roots, because a square root is the power one half:

```
>>> 9 ** 0.5
3.0
```

Precedence

● MUST MEMORISE

ORDER OF OPERATIONS

When several operators appear in one expression, Python applies them in this order, highest first:

ORDER	OPERATORS
1	() brackets
2	**
3	* / // %
4	+ -

Within one row, work left to right (except **, which goes right to left).

```
>>> 2 + 3 * 4
14
>>> (2 + 3) * 4
20
```

When in doubt, add brackets. They cost nothing and they make your intention unmistakable to a marker.

Comparison operators

● MUST MEMORISE

THESE GIVE TRUE OR FALSE

OPERATOR	MEANS	EXAMPLE	RESULT
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	7 > 5	True
<	Less than	4 < 10	True
>=	Greater than or equal to	7 >= 7	True
<=	Less than or equal to	2 <= 4	True

⚠ TRAP

= AGAINST ==

= assigns. **== asks.** They are unrelated, and confusing them is the most common single error in this course's error-spotting questions.

```
age = 18      # assigns 18 to age
age == 18     # asks: is age 18? Gives True.
```

Write `if age = 18:` and Python stops you with a `SyntaxError`, because a question was expected and you gave it an instruction.

Python lets you write a range check the way mathematics does, and it means what it looks like:

```
>>> bmi = 22.4
>>> 18.5 <= bmi <= 24.9
True
```

This is worth remembering: a question in this course asks for exactly this test on a BMI value, and the chained form is shorter and clearer than joining two comparisons with `and`.

Logical operators

● MUST MEMORISE

AND, OR, NOT

OPERATOR	TRUE WHEN	EXAMPLE	RESULT
<code>and</code>	Both sides are true	True and False	False
<code>or</code>	Either side is true	True or False	True
<code>not</code>	The other way round	not True	False

These are English words in Python, not symbols. If you have seen `&&` and `||` elsewhere, they do not exist here.

```
>>> n = 15
>>> n % 3 == 0 and n % 5 == 0
True
```

Fix that line in your memory. It asks "is `n` divisible by both 3 and 5", and it is the first test in the FizzBuzz style question this course sets.

Assignment operators

● MUST MEMORISE

SHORTHAND

Changing a variable using its own current value is so common it has shorthand:

SHORTHAND	MEANS
<code>x += 1</code>	<code>x = x + 1</code>
<code>x -= 1</code>	<code>x = x - 1</code>
<code>x *= 2</code>	<code>x = x * 2</code>
<code>x /= 2</code>	<code>x = x / 2</code>

`x += 1` is read "increment `x`". You will write it in every counting loop in Module Three, and forgetting it is what makes a loop run forever.

Tracing a program by hand, one line at a time, writing down what each variable holds, is the skill this course tests as "what will the following code display". Practise it now on something small, so that it is automatic later.

```
a = 7
b = 2
a += b
c = a // b
d = a % b
print(a, b, c, d)
```

Take one line at a time and never look ahead:

LINE	A	B	C	D	OUTPUT
a = 7	7	-	-	-	
b = 2	7	2	-	-	
a += b	9	2	-	-	
c = a // b	9	2	4	-	
d = a % b	9	2	4	1	
print(...)	9	2	4	1	9 2 4 1

A dash means the variable does not exist yet. Using one before it exists is a NameError.

CONFIRMED AGAINST A REAL INTERPRETER

```
9 2 4 1
```

1. What is `10 / 2`, and what type is it?
2. How do you test whether `n` is divisible by 5?
3. What is `2 ** 3 ** 2`?
4. What is the difference between `=` and `==`?
5. What is `17 // 3` and `17 % 3`?

ANSWERS

1. `5.0`, a float. `/` always gives a float.
2. `n % 5 == 0`
3. `512`. `**` groups from the right, so it is `2 ** 9`, not `(2 ** 3) ** 2`.
4. `=` assigns a value to a name. `==` asks whether two values are equal and gives back True or False.
5. `5` and `2`. Seventeen is three fives remainder two.

MODULE TWO · UNIT 4

Working with Strings

Text, in depth. An entire question on your paper is nothing but this unit, so it earns the space.

By the end of this unit you can: index and slice a string, explain why a string cannot be changed, use every string method this course examines, and write the text analysis programs it sets.

A string is a sequence

● MUST MEMORISE

INDEXING AND SLICING

A string is a sequence of characters, and everything you learned about indexing a list in Module Three applies here unchanged. The first character is number 0.

```
>>> name = "Python"
>>> name[0]
'p'
>>> name[-1]
'n'
>>> name[0:3]
'Pyt'
>>> len(name)
6
```

Negative indexes count from the right. Slices exclude the stop. Same rules, same traps, and `name[6]` is an `IndexError` just as a list's would be.

A list can be changed in place. **A string cannot.** This property is called being **immutable**, and it is asked directly.

```
>>> name = "Python"
>>> name[0] = "J"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

Compare that message with the tuple's in Module Three: they are the same sentence with a different type name, because they are the same idea.

The consequence, and this is the important part: every string method that looks like it changes a string actually **returns a new string** and leaves the original untouched.

```
>>> name = "Python"
>>> name.upper()
'PYTHON'
>>> name
'Python'
```

`name` is **unchanged**. To keep the result you must catch it:

```
>>> name = name.upper()
>>> name
'PYTHON'
```

Calling `name.upper()` on its own line and expecting `name` to change is one of the most common errors in this course. It is the mirror image of the `sort()` trap from Module Three: a list method changes the list and returns `None`, a string method changes nothing and returns the answer.

The methods

● MUST MEMORISE

EVERY ONE OF THESE IS EXAMINED

METHOD	DOES	EXAMPLE	GIVES
<code>upper()</code>	All capitals	<code>"ada".upper()</code>	<code>'ADA'</code>
<code>lower()</code>	All small letters	<code>"ADA".lower()</code>	<code>'ada'</code>
<code>strip()</code>	Removes whitespace from both ends	<code>" hi ".strip()</code>	<code>'hi'</code>
<code>replace(a, b)</code>	Every a becomes b	<code>"a-b".replace("-", " ")</code>	<code>'a b'</code>
<code>split(sep)</code>	Cuts into a list	<code>"a,b".split(",")</code>	<code>['a', 'b']</code>
<code>index(sub)</code>	Where sub starts	<code>"Ada".index("d")</code>	1
<code>find(sub)</code>	Same, but -1 if absent	<code>"Ada".find("z")</code>	-1
<code>count(sub)</code>	How many times	<code>"banana".count("a")</code>	3
<code>startswith(s)</code>	True or False	<code>"https://x".startswith("https")</code>	True
<code>isalpha()</code>	True only if every character is a letter	<code>"Ada".isalpha()</code>	True
<code>join(list)</code>	Glues a list into a string	<code>", ".join(["a", "b"])</code>	<code>'a, b'</code>

A question asks you to "list and briefly explain four common string methods, with an example of each". The four safest are `upper`, `lower`, `strip` and `replace`: each is one line to explain and impossible to get wrong.

● MUST MEMORISE

INDEX VS FIND

They do the same job and fail differently, which is the whole reason both exist.

```
>>> "Ada".find("z")
-1
>>> "Ada".index("z")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: substring not found
```

find returns -1 when it is not there. **index** raises a **ValueError**. Use **find** when absence is normal, **index** when absence means something has gone wrong.

`split()` behaves differently depending on whether you give it a separator, and both forms are examined.

```
>>> "70 55 90".split()
['70', '55', '90']
>>> "ada@mail.com".split("@")
['ada', 'mail.com']
```

- ▶ **No argument:** cuts at every run of whitespace, and ignores extra spaces. This is the one used for "values separated by spaces".
- ▶ **With an argument:** cuts at exactly that text, which is then thrown away.

It always returns a list of strings, never numbers. That is the split-and-convert pattern from Module Three: `split` then `int`.

Note what happens when the separator is not there at all. You get a list of one:

```
>>> "nothing here".split("@")
['nothing here']
```

No error. So `parts[1]` on that would be an `IndexError`, which is worth remembering when a question asks you to split an email at the `@`.

Meant to extract the first name and put it in capitals. Find the errors:

```
full_name = "Ada Lovelace"
first_name = full_name[0:full_name.index()]
print("First name:", first_name.uppercase())
```

Two errors, and both are invented method calls:

- ▶ `index()` is called with **nothing to look for**. It needs the text you want the position of. Here that is the space.
- ▶ `uppercase()` **does not exist in Python**. The method is `upper()`. Nothing in the language is called `uppercase`.

Only the first speaks, because the program stops there:

```
TypeError: index expected at least 1 argument, got 0
```

Fix that, run again, and only then do you meet the second:

```
AttributeError: 'str' object has no attribute 'uppercase'
```

Note what you do **not** get here: a suggestion. Python offers "Did you mean" only when your typo is close enough to a real name. Write `uppe()` and it says *Did you mean: 'upper'?*; write `uppercase()` and it just says no. **The help arrives when you least need it.** Do not rely on it.

The corrected program:

```
full_name = "Ada Lovelace"
first_name = full_name[0:full_name.index(" ")]
print("First name:", first_name.upper())
```

```
First name: ADA
```

`full_name.index(" ")` is **3**, the position of the space, so the slice takes characters 0 to 2. The space itself is excluded, because a slice stops before its stop.

A neater answer, worth mentioning to a marker: `full_name.split()[0]` does the same job in fewer characters and does not break when there is no space.

A "what is the output, explain each line" question. It looks gentle and has a trap in the middle.

```
text = "  Hello, Python!  "
print(text.strip())
print(text.lower().replace("python", "world"))
print(len(text))
```

```
Hello, Python!
  hello, world!
18
```

Line by line, which is what "explain the result of each line" wants:

- ▶ `text.strip()` removes the two spaces at each end, giving `Hello, Python!`. **text itself is unchanged**, which the next line proves.
- ▶ `text.lower()` gives `" hello, python! "`. **Now** `replace("python", "world")` matches, because the text is lowercase by the time `replace` sees it. The result keeps its spaces, because `strip` on the line before returned a new string and changed nothing.
- ▶ `len(text)` is **18**, counting the original with all four spaces: 2 at the front, 14 for `Hello, Python!`, 2 at the back.

The trap is the middle line. Answer it without noticing the `lower()` and you will say `replace` finds nothing and the text is unchanged. It does match, and the answer is `hello, world!`. Chained methods run left to right, and each one works on what the one before it returned.

Process a list of student email addresses and print the usernames, where the username is the part before the @.

- 1 Put the addresses in a list.
- 2 Loop through it.
- 3 Split each at "@" and take item 0.
split gives a list; [0] is before the @, [1] is after
- 4 Print in the format the question shows.

EMAILS.PY

```
emails = [  
    "azu.ezenwoke@university.edu.ng",  
    "odunayo.osofuye@university.edu",  
    "emmanuel.franklin@university.edu",  
    "ada.lovelace@university.edu",  
    "bola.ahmed@university.edu",  
]  
  
for email in emails:  
    username = email.split("@")[0]  
    print(f>Email: {email} | Username: {username})
```

```
Email: azu.ezenwoke@university.edu.ng | Username: azu.ezenwoke  
Email: odunayo.osofuye@university.edu | Username: odunayo.osofuye  
Email: emmanuel.franklin@university.edu | Username: emmanuel.franklin  
Email: ada.lovelace@university.edu | Username: ada.lovelace  
Email: bola.ahmed@university.edu | Username: bola.ahmed
```

Now you try: the question asks for "at least five addresses". Why does `split("@")[0]` still work on an address with a dot in the domain, such as `university.edu.ng`? (Because it splits on @, not on dots. The domain's dots are all in item 1, which we never look at.)

The extension: read a paragraph, split it into sentences at `"."`, and for each one strip it, skip it if empty, count the words, put it in capitals, and count the vowels.

- 1 Split at `"."`. This is where the empty pieces come from: a paragraph ending in a full stop leaves an empty string after the last one.
"A. B.".split(".") gives ['A', ' B', '']
- 2 `strip()` each piece, then **skip the empties**. That is what the requirement is protecting you from.
- 3 `split()` with no argument to count words.
- 4 Count vowels by walking the characters and testing membership with `in`.

ANALYSE.PY

```
paragraph = input("Enter a paragraph: ")

for sentence in paragraph.split("."):
    sentence = sentence.strip()
    if sentence == "":
        continue

    words = len(sentence.split())

    vowels = 0
    for ch in sentence.lower():
        if ch in "aeiou":
            vowels += 1

    print(f"Sentence: {sentence.upper()}")
    print(f" Words: {words}, Vowels: {vowels}")
```

Entering Python is fun. I like code.

```
Enter a paragraph: Python is fun. I like code.
Sentence: PYTHON IS FUN
Words: 3, Vowels: 3
Sentence: I LIKE CODE
Words: 3, Vowels: 5
```

Three details that decide whether this works:

- ▶ `ch in "aeiou"` asks whether that character appears in the string `"aeiou"`. It is a neat way to test membership without five comparisons joined by `or`.
- ▶ **Count vowels on `sentence.lower()`**, not on the sentence. The `I` in "I like" is a vowel, and testing it against lowercase `"aeiou"` would miss it. Count first, then display in capitals.
- ▶ **"Python is fun" has three vowels, not four.** They are the `o`, the `i` and the `u`. The `y` in Python is *not* counted, because the question defines the vowels as (a, e, i, o, u) and `y` is not in that list. Follow the question's definition, not your instinct about English.

Now you try: why is the `continue` needed at all? (The paragraph ends with a full stop, so splitting leaves an empty final piece. Without the skip you would print a blank sentence with 0 words.)

Ask for the **consonants** instead and that loop quietly gives the wrong answer. A consonant is a letter that is not a vowel, so it is tempting to flip the test to `ch not in "aeiou"`. But the space is not in "aeiou" either, and neither is the full stop, so both would be counted as consonants.

You have to ask the other half of the question: **is this character a letter at all?** That is `isalpha()`, and it is true only when every character it is given is a letter.

```
>>> "Ada".isalpha()
True
>>> "Ada Lovelace".isalpha()
False
>>> " ".isalpha()
False
>>> "!".isalpha()
False
```

The space makes the whole of "Ada Lovelace" false. That is exactly the behaviour you want one character at a time, so a consonant is both things at once:

```
consonants = 0
for ch in sentence.lower():
    if ch.isalpha() and ch not in "aeiou":
        consonants += 1
```

On "Python is fun" that gives **8**: the p, y, t, h, n, s, f and n. Eleven letters, minus the three vowels. The y is counted, because the question defines the vowels as a, e, i, o, u and y is not one of them: the same rule that makes the vowel count 3 and not 4 makes the consonant count 8.

1. What does **immutable** mean, and what follows from it for `name.upper()` ?
2. Difference between **find** and **index** ?
3. What is `"a b".split()` against `"a-b".split("-")` ?
4. What is `len(" Hello, Python! ")` ?
5. Which method does not exist: **upper** or **uppercase** ?

ANSWERS

1. A string cannot be changed in place. So `name.upper()` returns a new string and leaves `name` alone. You must write `name = name.upper()` to keep it.
2. Both give the position of the text. `find` returns -1 when it is absent; `index` raises a `ValueError`.
3. Both give `['a', 'b']`. No argument splits at whitespace; an argument splits at exactly that text.
4. **18**. Two spaces, 14 characters, two spaces.
5. `uppercase` does not exist. The method is `upper`.

MODULE THREE

Control Flow and Data Structures

Choosing, repeating, and storing many things at once. Two of the six questions on your paper come from this module alone, and the habits you build here decide how the other four go.

```
if n % 2 == 0:
    print( "even" )
else:
    print( "odd" )
```

Decision Making in Python

Making the program choose. Everything so far has run top to bottom with no choices at all.

By the end of this unit you can: write `if`, `if/else` and `if/elif/else`, nest them, explain why order matters, and write the divisibility program this course sets every year.

if

TEACH

An `if` statement runs a block only when a condition is true. You met the shape in Module Two; here is the full rule.

```
if condition:
    # this block runs only if the condition is True
    print("yes")
```

Three parts, and all three are compulsory:

- ▶ The word `if`.
- ▶ A **condition**: anything that produces `True` or `False`. Usually a comparison, as in Module Two, Unit 3.
- ▶ A **colon**, then an indented block beneath.

```
>>> age = 20
>>> if age >= 18:
...     print("You may vote.")
...
You may vote.
```

At the prompt, a block makes the prompt change to `...` to show it is waiting for more. Press **Enter** on an empty line to finish the block and run it.

if / else

TEACH

`else` gives the other path. Exactly one of the two blocks runs, always.

```
n = 7
if n % 2 == 0:
    print("even")
else:
    print("odd")
```

```
odd
```

`else` takes no condition of its own. It means "in every other case", so writing a condition after it is an error.

if / elif / else

● MUST MEMORISE

THE THREE WAY CHOICE

For more than two paths, use `elif`, short for "else if". You may have as many as you like. Python tests them **in order, from the top**, and stops at the first one that is true. Every later test is skipped, even if it would also be true.

```
score = 72
if score >= 70:
    print("A")
elif score >= 60:
    print("B")
elif score >= 50:
    print("C")
else:
    print("F")
```

A

72 is greater than 70, so `A` prints and nothing else is tested. Note that 72 is *also* greater than 60 and 50. Those tests never ran, and that is the whole point of `elif`.

⚠ TRAP

ORDER DECIDES THE ANSWER

Write the same tests in the wrong order and every mark goes:

```
score = 72
if score >= 50:
    print("C")
elif score >= 70:
    print("A")
```

C

72 passes the first test, so it prints `C` and stops. The `A` branch is unreachable: no score can ever print `A`, because anything that is 70 or more is also 50 or more. **With `elif`, put the narrowest test first.**

Using separate `if` statements instead of `elif` is a different bug with the same smell: each one is tested independently, so a score of 72 would print `A`, `B` and `C`.

An `if` may live inside another. Each level indents four more spaces.

```
n = 15
if n > 0:
    if n % 2 == 0:
        print("positive and even")
    else:
        print("positive and odd")
else:
    print("not positive")
```

positive and odd

Read the indentation to see which `else` belongs to which `if`: an `else` pairs with the `if` at its own indent level. The inner `else` is at 4 spaces, so it answers the inner `if`. The outer one is at 0, so it answers the outer `if`.

Ask the user for a number of exercise sessions. If it is not positive, say so. Otherwise: divisible by both 3 and 5, print "Active - FizzBuzz"; by 3 only, "Active - Fizz"; by 5 only, "Active - Buzz"; otherwise "Delulu - Not divisible by 3 or 5".

- 1 Read the number and convert it, because `input` gives a string.
`n = int(input(...))`
- 2 Check positive first, and put everything else inside that branch. The requirement says "check if the number is positive" before any of the rest.
- 3 **Test "both 3 and 5" first.** This is the trap above. 15 is divisible by 3, so a "divisible by 3" test placed first would catch 15 and never reach FizzBuzz.
narrowest test first
- 4 Then 3 alone, then 5 alone, then the catch all `else`.

FITNESS.PY

```
n = int(input("Number of sessions: "))

if n > 0:
    if n % 3 == 0 and n % 5 == 0:
        print("Active - FizzBuzz")
    elif n % 3 == 0:
        print("Active - Fizz")
    elif n % 5 == 0:
        print("Active - Buzz")
    else:
        print("Delulu - Not divisible by 3 or 5")
else:
    print("Please enter a positive number")
```

Run it four times, with 15, 9, 10 and 7:

```
Number of sessions: 15
Active - FizzBuzz
Number of sessions: 9
Active - Fizz
Number of sessions: 10
Active - Buzz
Number of sessions: 7
Delulu - Not divisible by 3 or 5
```

Now you try: what does it print for 0, and for 30? (0 is not positive, so "Please enter a positive number". 30 is divisible by both, so "Active - FizzBuzz".)

1. In an `if/elif/elif/else`, how many blocks run?
2. Why must "divisible by 3 and 5" be tested before "divisible by 3"?
3. How do you tell which `if` an `else` belongs to?

ANSWERS

1. Exactly one. Python stops at the first true test.
2. Because anything divisible by both is also divisible by 3. The wider test would catch it first and the narrower branch could never run.
3. By indentation. An `else` pairs with the `if` at the same indent level.

MODULE THREE • UNIT 2

Loops: Repetition and Branching

Doing something many times. This unit contains the two patterns that carry a quarter of the marks on your paper.

By the end of this unit you can: write `while` and `for` loops, use `range`, use `break` and `continue`, explain `for/else`, avoid infinite loops, and write both of the "accept any number of inputs" patterns.

while

TEACH

A `while` loop repeats its block **as long as** a condition stays true. It checks before each pass, so a condition false at the start means the block never runs at all.

```
i = 1
while i <= 3:
    print(i)
    i += 1
```

```
1
2
3
```

Three parts, and leaving out the third is the classic disaster:

- ▶ **Set up** before the loop: `i = 1`.
- ▶ **Test** in the header: `i <= 3`.
- ▶ **Change** inside the block: `i += 1`. Without this, `i` stays 1 forever and the condition never turns false.

● MUST MEMORISE

TWO KINDS OF LOOP

This distinction is asked directly, in those words:

	CONDITION CONTROLLED	COUNT CONTROLLED
Written with	while	for
Repeats	While something stays true	Once per item in a sequence
Use when	You do not know how many times	You know how many times
Example	Read until the user types 'done'	Read exactly five scores

for and range

TEACH

A `for` loop walks through a sequence, giving you one item at a time. The commonest sequence is made by `range`.

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

Read it as: "for each value `i` in this sequence, do the block".

● MUST MEMORISE

RANGE

`range` starts at 0 and stops *before* the number you give. This off by one is deliberate and constant, and it is worth marks every year.

WRITTEN	GIVES	HOW MANY
<code>range(5)</code>	0, 1, 2, 3, 4	5
<code>range(1, 5)</code>	1, 2, 3, 4	4
<code>range(1, 10, 2)</code>	1, 3, 5, 7, 9	5
<code>range(5, 0, -1)</code>	5, 4, 3, 2, 1	5

One argument is the stop. Two are start and stop. Three are start, stop and **step**. The stop is never included, in every case.

So to count 1 to 10 inclusive you must write `range(1, 11)`.

break and continue

● MUST MEMORISE

THE DIFFERENCE

break leaves the loop entirely. Nothing further repeats.

continue skips the rest of this pass only, and goes straight back to the top for the next one.

```
>>> for i in range(5):  
...     if i == 3:  
...         break  
...     print(i)  
...  
0  
1  
2
```

```
>>> for i in range(5):  
...     if i == 3:  
...         continue  
...     print(i)  
...  
0  
1  
2  
4
```

Same loop, one word different. **break** stops at 3 and never prints 4. **continue** skips only 3 and carries on to 4.

This is the single most instructive broken snippet in the course. It is presented as "intended to print all even numbers from 1 to 10, skipping number 6", and you are asked to find the error.

```
i = 1
while i <= 10:
    if i % 2 == 0:
        if i == 6:
            continue
        print(i)
    i += 1
```

It does not merely print the wrong numbers. **It hangs forever.** It prints 2, then 4, and then stops responding.

Trace it. When `i` is 6, `continue` jumps back to the top of the `while` immediately, which means it **skips** `i += 1` on its way past. So `i` is still 6 at the next test, and 6 again, forever.

The lesson, and it is general: in a `while` loop, `continue` skips everything below it, including the line that moves the loop forward. In a `for` loop this cannot happen, because the `for` advances the counter itself.

The safe fix removes the need for `continue` at all:

```
i = 1
while i <= 10:
    if i % 2 == 0 and i != 6:
        print(i)
    i += 1
```

```
2
4
8
10
```

If a question insists you keep `continue`, then `i += 1` must come *before* it, and you must be careful not to skip the increment.

for / else

● MUST MEMORISE

UNUSUAL AND EXAMINED

A loop may have an `else`. This surprises nearly everyone, because it does not mean what it looks like.

The `else` block runs when the loop finishes normally. It is skipped if the loop was stopped by `break`.

Read `else` here as "if no break happened".

```
>>> for i in range(5):
...     if i == 3:
...         break
...     print(i)
... else:
...     print("Loop finished")
...
0
1
2
```

"Loop finished" never prints. The `break` at 3 stopped the loop, so the `else` was skipped. Remove the `break` and it would print after 0 1 2 3 4.

This exact snippet has been set as a "what is the output" question. The wrong answer, and the one most candidates give, is to include "Loop finished".

Accepting any number of inputs

Part (e) of every question on the most recent paper asks you to extend a fixed size program to handle any number of inputs. There are exactly two patterns, and between them they are worth a quarter of the marks you can attempt. Learn both cold.

Keep asking until the user types a particular word. That word is called a **sentinel**. Here it is `done`.

```
names = []
while True:
    name = input("Name (or 'done'): ")
    if name == "done":
        break
    names.append(name)

print("You entered:", names)
```

```
Name (or 'done'): Ada
Name (or 'done'): Bola
Name (or 'done'): done
You entered: ['Ada', 'Bola']
```

The shape is always the same:

- ▶ `while True:` means "loop forever". It is not a mistake here, because the `break` is the exit.
- ▶ **Test for the sentinel first**, before doing anything with the value. Otherwise 'done' gets stored as a name.
- ▶ `break` leaves the loop.

Note that the sentinel is checked with `==` against a string, so it is case sensitive: 'Done' would not stop this loop.

Take one line holding many values separated by spaces, and turn it into a list of numbers.

```
line = input("Enter scores separated by spaces: ")
parts = line.split()
scores = []
for p in parts:
    scores.append(int(p))

print(scores)
```

```
Enter scores separated by spaces: 70 55 90
[70, 55, 90]
```

`split()` with no argument cuts a string at every run of whitespace and gives back a **list of strings**. It does not convert anything, which is why the loop and `int()` are still needed.

```
>>> "70 55 90".split()
['70', '55', '90']
```

Those are strings, in quotes. Forget the conversion and you will be sorting "90" before "70" alphabetically, or joining them end to end instead of adding.

1. What does `range(1, 5)` give?
2. Difference between `break` and `continue`?
3. When does a loop's `else` block run?
4. Why can `continue` hang a `while` loop but not a `for` loop?
5. What does `"70 55".split()` give, and what type are the items?

ANSWERS

1. 1, 2, 3, 4. The stop value is never included.
2. `break` leaves the loop completely. `continue` skips the rest of this pass and goes to the next one.
3. When the loop finishes normally. It is skipped if `break` stopped the loop. Read it as "if no break happened".
4. Because `continue` skips the rest of the block, including the line that increments the counter. A `for` loop advances its own counter, so nothing can skip it.
5. `['70', '55']`. They are strings, and must be converted with `int()` before arithmetic.

MODULE THREE • UNIT 3

Lists and Tuples

Holding many values under one name.

By the end of this unit you can: create and index lists, slice them, use the list methods this course examines, explain negative indexing, and say what a tuple is and how it differs from a list.

Lists

TEACH

A **list** holds many values in order, under one name. Write it with square brackets, values separated by commas.

```
>>> scores = [70, 55, 90, 45]
>>> scores
[70, 55, 90, 45]
>>> len(scores)
4
```

A list can hold anything, including a mixture, and including other lists. An empty list is `[]`, which is where most programs start.

● MUST MEMORISE

INDEXING STARTS AT 0

Reach one item with square brackets. **The first item is number 0**, not 1.

```
>>> scores = [70, 55, 90, 45]
>>> scores[0]
70
>>> scores[3]
45
```

So a list of 4 items has indexes 0, 1, 2, 3. **The last index is always len minus 1.** Asking for `scores[4]` is the commonest runtime error in this course:

```
>>> scores[4]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
```

Negative indexes count from the right, starting at `-1` for the last item. This is the neat way to get the end without knowing the length:

```
>>> scores[-1]
45
>>> scores[-2]
90
```

TEACH

LISTS CAN BE CHANGED

Unlike a string, a list can be modified in place. This property is called being **mutable**.

```
>>> scores = [70, 55, 90, 45]
>>> scores[1] = 65
>>> scores
[70, 65, 90, 45]
```

Slicing

● MUST MEMORISE

SLICES

A **slice** takes several items at once, written `[start:stop]`. As with `range`, **the stop is not included**.

```
>>> nums = [1, 2, 3, 4, 5]
>>> nums[1:3]
[2, 3]
>>> nums[:2]
[1, 2]
>>> nums[2:]
[3, 4, 5]
>>> nums[-3:]
[3, 4, 5]
```

Leave out the start to mean "from the beginning". Leave out the stop to mean "to the end". A slice always gives back a **new list**; the original is untouched.

A **method** is a function that belongs to a value, called by writing a dot after it. These are the ones this course uses:

METHOD	DOES	NOTE
<code>append(x)</code>	Adds x to the end	The one you use in every loop
<code>insert(i, x)</code>	Puts x at index i	Shifts the rest right
<code>remove(x)</code>	Removes the first x by value	ValueError if x is absent
<code>pop(i)</code>	Removes and returns the item at i	No i means the last
<code>index(x)</code>	Gives the index of the first x	ValueError if x is absent
<code>sort()</code>	Sorts the list in place	Returns None
<code>reverse()</code>	Reverses in place	Returns None

And three built in functions that take a list: `len()` , `max()` , `min()` , plus `sum()` for numbers.

⚠ TRAP

SORT() RETURNS NONE

`sort()` changes the list and gives back **nothing**. So assigning its result destroys your list:

```
>>> scores = [70, 55, 90]
>>> scores = scores.sort()
>>> print(scores)
None
```

Correct is to call it on its own line:

```
>>> scores = [70, 55, 90]
>>> scores.sort()
>>> scores
[55, 70, 90]
```

To sort the other way, pass `reverse=True` :

```
>>> scores.sort(reverse=True)
>>> scores
[90, 70, 55]
```

If you want a sorted copy and the original kept, use `sorted(scores)` , which *does* return a new list.

⚠ TRAP

REMOVE TAKES A VALUE, NOT AN INDEX

`remove` is given the **value** to delete. `pop` is given the **index**. Confusing them is a set error in this course:

```
>>> items = ["apples", "bananas", "carrots"]
>>> items.remove("banana")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

The value is `"bananas"`, not `"banana"`. The message says exactly that: `x` is not in the list.

TEACH

+ AND * ON LISTS

`+` joins two lists into a new one. `*` repeats a list.

```
>>> [1, 2] + [3, 4]
[1, 2, 3, 4]
>>> [1, 2] * 2
[1, 2, 1, 2]
```

Both appear in "what is the output" questions. Note `[1, 2] * 2` is `[1, 2, 1, 2]`, not `[2, 4]`: it repeats the list, it does not multiply the numbers.

Tuples

● MUST MEMORISE

LIST VS TUPLE

A **tuple** is a list that cannot be changed. Write it with round brackets.

```
>>> point = (3, 4)
>>> point[0]
3
>>> point[0] = 9
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

	LIST	TUPLE
Written with	[]	()
Can be changed?	Yes, it is mutable	No, it is immutable
Has append, remove, sort?	Yes	No
Indexing and slicing	Yes	Yes, both work the same
Use for	Data that changes	Data that must not change

Indexing and slicing work identically on both. Only *changing* is forbidden.

```
nums = [1, 2, 3, 4, 5]
total = 0
for n in nums:
    if n % 2 == 0:
        continue
    total += n
print(total)
```

The `continue` skips the even numbers, so only the odd ones are added. Take it one pass at a time:

PASS	N	N % 2 == 0?	ACTION	TOTAL
start	-	-	-	0
1	1	False	add	1
2	2	True	skip	1
3	3	False	add	4
4	4	True	skip	4
5	5	False	add	9
print	-	-	-	9

CONFIRMED AGAINST A REAL INTERPRETER

9

Note this `continue` is safe: it is in a `for` loop, which advances its own counter. The same code as a `while` loop would need care.

1. In `nums = [1,2,3,4,5]` , what is `nums[-2]` ?
2. What is `nums[1:3]` ?
3. What does `scores = scores.sort()` leave in `scores` ?
4. Give two differences between a list and a tuple.
5. What is `[1, 2] * 2` ?

ANSWERS

1. 4 . Negative indexes count from the right, starting at -1.
2. [2, 3] . The stop index is not included.
3. None . `sort()` sorts in place and returns nothing. Call it on its own line.
4. A list uses square brackets and is mutable; a tuple uses round brackets and is immutable. A list has `append` , `remove` and `sort` ; a tuple has none of them.
5. [1, 2, 1, 2] . It repeats the list.

Sets and Dictionaries

The other two built in data structures. One removes duplicates for free; the other looks things up by name.

By the end of this unit you can: create sets, use them to remove duplicates, apply the four set operations, create and read dictionaries, change them, and explain what happens when a dictionary holds a list.

Sets

● MUST MEMORISE

A SET HAS NO DUPLICATES

A **set** is an unordered collection with **no duplicates**. Write it with curly brackets, or build one from a list with `set()`.

```
>>> students = ["Alice", "Bob", "Alice", "Eve", "Bob"]
>>> len(set(students))
3
```

Five names in, three unique out, with no loop and no test. **This is the standard way to remove duplicates and it is set as an examination question.**

Two consequences of being a set, both examinable:

- ▶ **No order.** There is no first item, so `s[0]` is an error.
- ▶ **No duplicates, ever.** Adding something already present does nothing.

To see the contents in a form you can rely on, sort them. `sorted()` gives back a **list**, in order:

```
>>> sorted(set(students))
['Alice', 'Bob', 'Eve']
```

Note the square brackets in that answer: `sorted` handed back a list, not a set. That is what makes the order meaningful.

"Unordered" is stronger than it sounds. Printing a set of strings gives an order that **varies from one run to the next**, on the same machine, with the same code. Running this same line in three fresh sessions:

```
{'Alice', 'Eve', 'Bob'}
{'Bob', 'Alice', 'Eve'}
{'Alice', 'Bob', 'Eve'}
```

Python deliberately randomises how it stores text each time it starts, so the order a set happens to print in is an accident of that run. It is not alphabetical, it is not insertion order, and it is not stable.

What this means for you: never write a set's printed order as your answer, and never rely on it in a program. If a question asks what `print(set(...))` displays, the honest answer names the members and says the order is not defined. If you need a definite order, use `sorted()`.

Sets of small whole numbers do tend to print in ascending order, which is why `{1, 2, 3}` looks so well behaved. That is a side effect of how integers are stored, not a promise, so do not lean on it either.

This is presented as code that "should remove duplicates and print the number of unique students". Find the issue:

```
students = ["Alice", "Bob", "Alice", "Eve", "Bob"]
students = list(students)
unique_students = students
print("Total unique students:", len(unique_students))
```

```
Total unique students: 5
```

There is no error message. It runs perfectly and prints the wrong number, which is what makes it hard. Two things are wrong, and neither raises anything:

- ▶ `list(students)` makes a copy of a list that is already a list. It does nothing useful and removes no duplicates.
- ▶ `unique_students = students` just gives the same list a second name. Nothing was deduplicated.

The fix is one word:

```
students = ["Alice", "Bob", "Alice", "Eve", "Bob"]
unique_students = set(students)
print("Total unique students:", len(unique_students))
```

```
Total unique students: 3
```

● MUST MEMORISE

THE FOUR SET OPERATIONS

```
>>> a = {1, 2, 3, 4}
>>> b = {3, 4, 5, 6}
```

NAME	SYMBOL	METHOD	MEANS	RESULT
Union	$a \mid b$	<code>a.union(b)</code>	In either	{1, 2, 3, 4, 5, 6}
Intersection	$a \& b$	<code>a.intersection(b)</code>	In both	{3, 4}
Difference	$a - b$	<code>a.difference(b)</code>	In a but not b	{1, 2}
Symmetric difference	$a \wedge b$	<code>a.symmetric_difference(b)</code>	In one but not both	{1, 2, 5, 6}

Note `a - b` is not the same as `b - a`: difference has a direction. Here `b - a` is {5, 6}.

Dictionaries

● MUST MEMORISE

KEY AND VALUE

A **dictionary** stores pairs: a **key** and its **value**. You look things up by key rather than by position.

```
>>> person = {"name": "John", "age": 25}
>>> person["name"]
'John'
```

Curly brackets like a set, but with `key: value` pairs. The key goes in the square brackets, and it is **not** a number counting positions. There is no `person[0]` here.

Assigning to a key adds it if new, and replaces it if it exists:

```
>>> person["city"] = "Lagos"
>>> person["age"] = 26
>>> person
{'name': 'John', 'age': 26, 'city': 'Lagos'}
```

Asking for a key that does not exist is an error:

```
>>> person["height"]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'height'
```

Three methods give you the parts, and each is examinable:

```
>>> person = {"name": "John", "age": 25}
>>> list(person.keys())
['name', 'age']
>>> list(person.values())
['John', 25]
>>> for k, v in person.items():
...     print(k, "is", v)
...
name is John
age is 25
```

METHOD	GIVES YOU	EXAMPLE	RESULT
keys()	Every key	list(person.keys())	['name', 'age']
values()	Every value	list(person.values())	['John', 25]
items()	Each key with its value	for k, v in person.items()	a pair each time round

A plain `for` over a dictionary gives you the **keys**, not the values. That catches people out, so say it to yourself now.

● MUST MEMORISE

A DICTIONARY HOLDING A LIST

A value can be any type, including a list. This exact code has been set as a "what is the output, and explain each stage" question:

```
person = {
    "name": "John",
    "age": 25,
    "skills": ["Python", "Java"]
}
person["skills"].append("C++")
print(person["skills"])
```

```
['Python', 'Java', 'C++']
```

Take the middle line apart, because that is what "explain each stage" is asking for:

- ▶ `person["skills"]` looks up the key and gives back **the list itself**, not a copy.
- ▶ `.append("C++")` is then called on that list, so the list inside the dictionary is changed.

No reassignment happened, and none was needed. The list is mutable, and you reached it through the dictionary.

1. How do you remove duplicates from a list in one step?
 2. Why is `s[0]` an error on a set?
 3. What is `{1,2,3}` & `{3,4}` ?
 4. What does a plain `for` over a dictionary give you?
 5. Name the four built in data structures and one use for each.
-

ANSWERS

1. Pass it to `set()` . Wrap that in `len()` to count them.
2. A set is unordered, so there is no item at a position. Indexing has no meaning.
3. `{3}` . & is intersection, the items in both.
4. The keys. Use `.values()` or `.items()` for the rest.
5. List (ordered, changeable: a class register). Tuple (ordered, fixed: map coordinates). Set (unique, unordered: the distinct courses a student takes). Dictionary (lookup by key: a student's name against their score).

MODULE FOUR

Functions, Modules and File Handling

Packaging work under a name, borrowing work somebody else has done, and making data outlive the program that made it. Questions Three and Five come from here.

```
def square(x):  
    return x * x
```

```
print(square( 4 ))    16 ■
```

Functions in Python

Giving a piece of work a name, so you can ask for it by name.

By the end of this unit you can: define and call a function, explain parameters against arguments, use `return`, name the two types of user defined function, use default parameters, and say why functions matter.

What a function is

TEACH

A **function** is a block of code with a name, which runs only when you ask for it. You have been using functions since page one: `print`, `len`, `int` and `input` are all functions.

Now you write your own.

```
def greet():
    print("Hello!")

greet()
```

```
Hello!
```

Two separate things happened, and keeping them apart matters:

- ▶ **Defining** the function, with `def`. This only tells Python the name and the recipe. **Nothing runs.**
- ▶ **Calling** the function, by writing its name with brackets. **Now** the block runs.

Define it once, call it as many times as you like. Delete the last line above and the program prints nothing at all, because a definition on its own does nothing.

● MUST MEMORISE

WHY FUNCTIONS MATTER

Asked directly as "why are functions important in programming". Four reasons:

- ▶ **Reuse.** Write it once, call it anywhere. No copying.
- ▶ **Readability.** A name says what a block does, so the program reads as steps rather than detail.
- ▶ **Maintenance.** Fix a bug in one place and every caller is fixed.
- ▶ **Decomposition.** A large problem splits into small named pieces, each testable on its own.

● MUST MEMORISE

BUILT IN VS USER DEFINED

	BUILT IN	USER DEFINED
Who wrote it	Comes with Python	You did, with def
Needs defining?	No, always available	Yes, before you call it
Examples	print, len, int, max, range	greet, square, is_healthy

Parameters and arguments

● MUST MEMORISE

THE TWO WORDS

A function can be handed values to work on.

```
def greet(name):          # name is a PARAMETER
    print("Hello,", name)

greet("Ada")              # "Ada" is an ARGUMENT
```

Hello, Ada

A parameter is the name in the definition. An argument is the value you pass in at the call. Same slot, two words, depending on which side you are standing on.

Several parameters are separated by commas, and the arguments line up with them by position:

```
>>> def add(a, b):
...     print(a + b)
...
>>> add(2, 3)
5
```

return

● MUST MEMORISE

THE TWO TYPES OF FUNCTION

This is asked in those words: **explain the two types of user defined function.**

1. A void function does something and hands nothing back. It has no `return`.

```
def greet(name):  
    print("Hello,", name)
```

2. A value returning function works something out and hands it back with `return`.

```
def square(x):  
    return x * x
```

The difference is what you can do with the call. A value returning function's call *is* a value, so it can go anywhere a value can:

```
>>> def square(x):  
...     return x * x  
...  
>>> square(4)  
16  
>>> y = square(4) + 1  
>>> y  
17
```

⚠ TRAP

PRINT IS NOT RETURN

These look alike and are not alike at all.

```
>>> def shout(x):  
...     print(x * 2)  
...  
>>> result = shout(5)  
10  
>>> print(result)  
None
```

`shout` displayed 10, then handed back **nothing**. So `result` is `None`.

A function with no `return` returns `None`. Always. `print` shows a value to a human; `return` gives a value back to the program. If a question says "returns Healthy", it means `return`, and `print` will not earn the mark.

Also: `return` ends the function at once. Anything written after it in the same block never runs.

Meant to define a function that squares a number and call it. Find the errors:

```
def square(x)
    return x * x
print(Square(4))
```

Two errors, and you only ever hear about the first:

- ▶ **No colon** after `def square(x)`. This is a `SyntaxError`, caught while reading the file, so nothing runs.
- ▶ **Square with a capital S** does not exist. The function is `square`. Names are case sensitive.

Fix the colon and run again, and only then do you meet the second:

```
Traceback (most recent call last):
  File "C:\Users\you\sq.py", line 3, in <module>
    print(Square(4))
          ^^^^^
NameError: name 'Square' is not defined. Did you mean: 'square'?
```

That is the syntax before runtime rule from Module Two doing exactly what it promised. The corrected program:

```
def square(x):
    return x * x

print(square(4))
```

16

Default parameters

A parameter may be given a value to fall back on, used when the caller does not supply one.

```
>>> def greet(name="Guest"):
...     print("Hello,", name)
...
>>> greet()
Hello, Guest
>>> greet("Ada")
Hello, Ada
```

Called empty, `name` takes its default. Called with an argument, the argument wins. Note the comma and space in the output: that is `print` putting one space between its two items, not part of the string.

The rule: parameters with defaults must come after those without. Writing `def f(a=1, b)` is a `SyntaxError`, because Python could not tell what a single argument was meant to fill.

Define `is_healthy(bmi)` returning "Healthy" if the BMI is between 18.5 and 24.9, and "Unhealthy" otherwise. Read three patients and display each result as `Patient: John | BMI: 22.4 | Status: Healthy`.

- 1 **Return, do not print.** The question says "returns", so the function hands the word back and the caller displays it.
`return "Healthy"`
- 2 The range test is a chained comparison, from Module Two, Unit 3.
`18.5 <= bmi <= 24.9`
- 3 Loop three times with `range(3)`, reading a name and a BMI each pass. BMI is a decimal, so convert with `float`, not `int`.
- 4 Display with an f-string. The vertical bars are literal text.

HEALTH.PY

```
def is_healthy(bmi):
    if 18.5 <= bmi <= 24.9:
        return "Healthy"
    else:
        return "Unhealthy"

for i in range(3):
    name = input("Patient name: ")
    bmi = float(input("BMI: "))
    status = is_healthy(bmi)
    print(f"Patient: {name} | BMI: {bmi} | Status: {status}")
```

With John 22.4, Ada 17.9 and Bola 24.9:

```
Patient: John | BMI: 22.4 | Status: Healthy
Patient: Ada | BMI: 17.9 | Status: Unhealthy
Patient: Bola | BMI: 24.9 | Status: Healthy
```

Now you try: what does `is_healthy(24.95)` return, and why is that worth thinking about? ("Unhealthy", because 24.95 is above 24.9. The question's own boundary is 24.9, so anything between 24.9 and 25 is Unhealthy. Say the boundary you used and you cannot be marked wrong.)

The follow up asks for any number of patients, stopping at 'done'. That is the sentinel loop from Module Three, wrapped around the same function:

```
def is_healthy(bmi):
    if 18.5 <= bmi <= 24.9:
        return "Healthy"
    else:
        return "Unhealthy"

healthy = 0
total = 0
while True:
    name = input("Patient name (or 'done'): ")
    if name == "done":
        break
    bmi = float(input("BMI: "))
    status = is_healthy(bmi)
    total += 1
    if status == "Healthy":
        healthy += 1
    print(f"Patient: {name} | BMI: {bmi} | Status: {status}")

print(f"Total: {total}, Healthy: {healthy}, Unhealthy: {total - healthy}")
```

The function did not change at all. Only the loop around it did. **That is the point of a function, and it is the point of part (e).**

RECALL

COVER THE ANSWERS

1. Difference between a parameter and an argument?
2. Name the two types of user defined function.
3. What does a function without return give back?
4. Why does `print(Square(4))` fail when `square` is defined?

ANSWERS

1. A parameter is the name in the definition. An argument is the value passed at the call.
2. Void (returns nothing) and value returning (hands a value back with `return`).
3. None .
4. Names are case sensitive. `Square` and `square` are different names, and only one exists, so it is a `NameError`.

MODULE FOUR · UNIT 2

Modules and Packages

Using code somebody else already wrote.

By the end of this unit you can: import a module, use the two import forms, and name the modules this course uses.

- ▶ A **module** is a single file of Python code you can use from your own.
- ▶ A **package** is a folder of modules grouped together.
- ▶ The **standard library** is the large set of modules that ships with Python. Nothing to install; it is already there.

You use a module by **importing** it, and there are two forms:

```
>>> import math
>>> math.sqrt(16)
4.0
```

```
>>> from math import sqrt
>>> sqrt(16)
4.0
```

`import math` brings in the whole module, and you reach into it with a dot. `from math import sqrt` brings in that one name, which you then use bare. Both give `4.0`, a float, because `sqrt` always returns a float.

TEACH

THE ONES THIS COURSE USES

MODULE	GIVES YOU	EXAMPLE
math	Mathematics	<code>math.sqrt(16)</code> , <code>math.pi</code>
random	Random numbers	<code>random.randint(1, 6)</code>
sqlite3	Databases	Module Five
tkinter	Windows and buttons	Module Five

`random.randint(a, b)` is worth a note: **both ends are included**, unlike `range`. So `random.randint(1, 6)` can give 6.

MODULE FOUR · UNIT 3

File Handling in Python

Making data survive the program. Everything you have stored so far vanished the moment the program ended.

By the end of this unit you can: name the four access modes, open, write, read and close a file, explain reading against appending, use `with`, and write the record keeping programs this course sets.

The three steps

● MUST MEMORISE

ASKED IN THESE WORDS

Using a file is always three steps, and the question "state the three steps of file use" wants exactly these:

1. **Open** the file, which connects your program to it and says what you intend to do.
2. **Process** it: read from it, or write to it.
3. **Close** it, which finishes the job and makes sure everything you wrote is actually saved.

● MUST MEMORISE

THE FOUR ACCESS MODES

`open` takes the file name and the **mode**, a short string saying what you intend.

MODE	NAME	IF THE FILE EXISTS	IF IT DOES NOT
"r"	Read	Reads it. Cannot write.	FileNotFoundError
"w"	Write	Erases it completely , then writes	Creates it
"a"	Append	Adds to the end, keeps what is there	Creates it
"x"	Exclusive create	FileExistsError	Creates it

"r" is the default, so `open("f.txt")` means read.

● MUST MEMORISE

READING AGAINST APPENDING

Asked directly. Give the pair:

- ▶ **Reading** ("r") takes data *out* of the file and changes nothing. The file must already exist.
- ▶ **Appending** ("a") puts data *into* the file, at the end, keeping everything already there. It creates the file if needed.

And the distinction that actually loses data: "w" **erases the whole file the moment you open it**, before you write a single character. If you meant to add to a file and opened it with "w" , everything in it is already gone. "a" is the one that adds.

Writing

TEACH

```
f = open("note.txt", "w")
f.write("This is a note.")
f.close()
```

Two things surprise people about `write` :

- ▶ **It adds no newline.** `print` does; `write` does not. Two writes in a row land on the same line unless you put `\n` in yourself.
- ▶ **It only writes strings.** Passing a number is a `TypeError`, so convert with `str()` first.

```
f = open("names.txt", "w")
f.write("Ada\n")
f.write("Bola\n")
f.close()
```

gives a file holding two lines. Without the `\n` it would hold `AdaBola` on one line.

⚠ TRAP

THIS EXACT CODE HAS BEEN SET

Meant to write a message to `note.txt` . Find the errors:

```
file = open("note.txt", "w")
file.write("This is a note."
file.close
```

Two errors, and the first hides the second:

- ▶ The `write` line is **missing its closing bracket**. That is a `SyntaxError`, so the file never runs.
- ▶ `file.close` has **no brackets**, so it is not a call. It names the method and throws it away, doing nothing. The file is never closed.

The second is the nastier one, because it is silent: no error, no message, and possibly no data in the file. Fixed:

```
file = open("note.txt", "w")
file.write("This is a note.")
file.close()
```

Brackets mean "do it". No brackets means "the thing itself". The same rule catches `conn.cursor` against `conn.cursor()` in Module Five.

with

● MUST MEMORISE

THE SAFE WAY

Forgetting `close()` is easy, and a crash between opening and closing skips it entirely. `with` closes the file for you, whatever happens.

```
with open("example.txt", "a") as f:
    f.write("New line added.\n")

print("Done writing.")
```

Done writing.

Read it as: open the file, call it `f`, and when the indented block ends close it automatically. There is no `close()` to write and none to forget.

What this code does, if asked: it opens `example.txt` in append mode, creating it if absent; adds one line to the end without disturbing what is already there; closes it; then prints `Done writing.` to the screen. Note that **only** "Done writing." appears on screen. The written line went into the file, not to the display.

Reading

● MUST MEMORISE

THREE WAYS TO READ

Given a `names.txt` holding `Ada` and `Bola` on two lines:

METHOD	GIVES	RESULT HERE
<code>f.read()</code>	The whole file as one string	<code>'Ada\nBola\n'</code>
<code>f.readline()</code>	The next line only, as a string	<code>'Ada\n'</code>
<code>f.readlines()</code>	A list of all lines	<code>['Ada\n', 'Bola\n']</code>

The usual way is to loop over the file directly, which gives one line at a time and does not load a large file into memory at once:

```
with open("names.txt") as f:
    for line in f:
        print(line.strip())
```

Ada
Bola

Every line you read keeps its `\n` on the end. That is why `strip()` is there: without it you would print a blank line after each name, because the line's own newline and `print`'s newline both land.

Store student records (name and score) until 'done' is entered, in `students.txt`, one per line as `Name, Score`. Then read it back, print each record, and count passes and fails. **The pass mark is 45 and above.**

- 1 Sentinel loop to collect, writing as you go. Mode `"w"` because we are creating the file fresh.

Test for 'done' before asking for the score

- 2 `write` needs a string and adds no newline, so build the line with an f-string and put `\n` on the end yourself.

- 3 Close, then reopen in `"r"`. The question asks for this explicitly.

- 4 Split each line at the comma, convert the score, count.

45 and above passes

STUDENTS.PY

```
with open("students.txt", "w") as f:
    while True:
        name = input("Name (or 'done'): ")
        if name == "done":
            break
        score = int(input("Score: "))
        f.write(f"{name}, {score}\n")

passes = 0
fails = 0
with open("students.txt", "r") as f:
    for line in f:
        name, score = line.strip().split(", ")
        score = int(score)
        print(f"{name} scored {score}")
        if score >= 45:
            passes += 1
        else:
            fails += 1

print(f"Passes: {passes}, Fails: {fails}")
```

Entering John 76, Ada 45, Bola 30, then done:

```
John scored 76
Ada scored 45
Bola scored 30
Passes: 2, Fails: 1
```

Ada passes. The mark is 45 *and above*, so the test is `>=`, not `>`. Getting that boundary wrong is the difference between 2 passes and 1.

`name, score = ...split(", ")` unpacks a two item list into two names at once. It works because `split` returned exactly two pieces.

The paper asks you to count passes and fails but **never states the pass mark in its printed text**. It appears only as a handwritten note: 45 and above.

If you meet a question like this with no threshold given, **state the one you are using** in a comment and in your output. A marker cannot fault an assumption you declared. They can fault a number that appeared from nowhere.

RECALL

COVER THE ANSWERS

1. Name the four access modes and what each does to an existing file.
2. What does "w" do to a file that already has data?
3. What is wrong with `file.close`?
4. Why does reading a line usually need `strip()`?
5. What does `with` save you from?

ANSWERS

1. "r" read, fails if absent. "w" write, erases it. "a" append, adds to the end. "x" create, fails if it exists.
2. Erases it completely, the moment it is opened, before anything is written.
3. No brackets, so it is not a call. Nothing happens and the file is never closed. No error is reported.
4. Each line keeps the `\n` that ended it in the file.
5. Forgetting `close()`, and skipping it when something goes wrong part way through.

MODULE FOUR · UNIT 4

Exception Handling

Catching a runtime error instead of letting it kill the program.

By the end of this unit you can: explain what an exception is, use `try` and `except`, catch the right error, and use `finally`.

TEACH

An **exception** is a runtime error, in the Module Two sense: the line was grammatically fine, but doing it turned out to be impossible. Unhandled, it prints a traceback and the program stops dead.

You met this one already:

```
>>> int("xyz")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: 'xyz'
```

In a program that asks the user for a number, that is not a bug in your code. It is a user typing badly. The program should say so and carry on, not crash.

```
try:
    age = int(input("Your age: "))
    print("Next year you are", age + 1)
except ValueError:
    print("That is not a whole number.")
```

Run it and type xyz :

```
Your age: xyz
That is not a whole number.
```

No traceback, no crash. How it works:

- ▶ Python runs the `try` block.
- ▶ If nothing goes wrong, the `except` block is skipped entirely.
- ▶ If a `ValueError` happens, Python **abandons the rest of the `try` block at once** and runs the `except` block instead.

Note the second line inside the `try` never ran. That is why the block should hold only the risky work and what depends on it.

EXCEPTION	HAPPENS WHEN	SEEN IN
<code>ValueError</code>	<code>int("xyz")</code>	Module Two, Unit 2
<code>ZeroDivisionError</code>	<code>10 / 0</code>	Module Two, Unit 1
<code>IndexError</code>	<code>[1,2][5]</code>	Module Three, Unit 3
<code>KeyError</code>	<code>d["absent"]</code>	Module Three, Unit 4
<code>TypeError</code>	<code>"a" + 1</code>	Module Two, Unit 2
<code>NameError</code>	Using a name that does not exist	Module Four, Unit 1
<code>FileNotFoundError</code>	<code>open("gone.txt")</code>	Module Four, Unit 3

Every one of these has appeared in this manual before this page. That is the point of the table: you have met them all as accidents, and now you can catch them on purpose.

A bare `except:` catches every error there is, including the ones caused by your own bugs, and hides them:

```
try:
    age = int(input("Age: "))
except:
    print("Something went wrong.")
```

Misspell a name inside that `try` and you get "Something went wrong" instead of a `NameError` telling you exactly which line and which name. You have thrown away the most useful message on the screen.

Name the exception you expect. `except ValueError:` catches bad input and leaves your own mistakes visible.

The exam's programs all begin by reading a number. Combining this unit with the loops from Module Three gives input that cannot be broken:

```
while True:
    try:
        n = int(input("Enter a whole number: "))
        break
    except ValueError:
        print("Not a whole number. Try again.")

print("You entered", n)
```

Typing `xyz`, then `3.5`, then `7`:

```
Enter a whole number: xyz
Not a whole number. Try again.
Enter a whole number: 3.5
Not a whole number. Try again.
Enter a whole number: 7
You entered 7
```

The `break` is **inside the `try`, after the conversion**. So it is only reached when the conversion worked. Put it after the `try` block and the loop would exit even on bad input.

Note `3.5` is rejected: `int()` only accepts a whole number written out, as Module Two, Unit 2 showed.

Two optional extras complete the shape:

- ▶ `else` runs **only if no exception happened**. Same idea as the loop `else` from Module Three: "if nothing interrupted".
- ▶ `finally` runs **always**, exception or not. It is for cleaning up.

```
try:
    f = open("data.txt")
except FileNotFoundError:
    print("No such file.")
else:
    print("Opened it.")
    f.close()
finally:
    print("Finished trying.")
```

With no `data.txt` present:

```
No such file.
Finished trying.
```

`finally` is what `with` does for you automatically, which is why `with` is preferred for files.

1. What is an exception?
2. What happens to the rest of the `try` block when an exception fires on its first line?
3. Why is a bare `except:` a bad idea?
4. When does `finally` run?

ANSWERS

1. A runtime error: the line was grammatically valid but could not be carried out. Unhandled, it stops the program with a traceback.
2. It is abandoned. Python jumps straight to the matching `except`.
3. It catches your own bugs too and replaces their messages with yours, hiding exactly the information you need.
4. Always, whether an exception happened or not.

MODULE FIVE

Databases and GUI Development

Storing data so it can be searched rather than merely saved, and putting a window on the front of a program. Question Six comes from here.

```
import sqlite3
conn = sqlite3.connect("school.db" )
cur.execute("SELECT * FROM students" )
conn.commit()
```

Python and Databases

A file holds data. A database organises it, so you can ask questions of it.

By the end of this unit you can: say what a database is and why it is used, explain what `sqlite3` does, name the three SQL commands this course wants, and write the contact program it sets.

What a database is

● MUST MEMORISE

ASKED IN THESE WORDS

A **database** is an organised collection of data, stored so that it can be searched, changed and retrieved efficiently. Data lives in **tables**: a table has named **columns**, and each **row** is one record.

Why it is used in programming, which is the second half of the question:

- ▶ **It lasts.** Like a file, the data outlives the program.
- ▶ **It can be searched.** Ask for one record out of a million without reading the other 999,999. A text file makes you read the lot.
- ▶ **It has structure.** Columns have types, so a score cannot quietly become text.
- ▶ **It handles many users at once** without them overwriting each other.
- ▶ **It keeps data consistent**, through rules such as a primary key that forbid nonsense.

Compare with Module Four's `students.txt`: to find one student there you read every line and split every one. In a database you ask for that student.

● MUST MEMORISE

WHAT SQLITE3 IS

SQL (say "sequel", or spell it out) is the language you ask a database questions in. It is not Python. You write it as a *string* inside your Python and hand it over to be run.

SQLite is a small database that keeps everything in a single ordinary file on your disk. There is no server to install and no password.

sqlite3 is the Python module that lets your program talk to an SQLite database. It **ships with Python**, so there is nothing to install: it is part of the standard library from Module Four, Unit 2.

That is the answer to "what role does the `sqlite3` module play in Python": it is the built in interface between your Python code and an SQLite database file, letting you create tables, insert rows, and query them from Python.

The three SQL commands

● MUST MEMORISE

NAME THESE THREE

COMMAND	FUNCTION	EXAMPLE
CREATE TABLE	Makes a new table and names its columns	CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT)
INSERT INTO	Adds a row of data	INSERT INTO students (name) VALUES ('Ada')
SELECT	Fetches rows back out	SELECT * FROM students

Two more are worth a sentence each, in case the question asks for more than three: `UPDATE` changes rows that already exist, and `DELETE` removes rows.

SQL keywords are conventionally written in capitals. It is not required, but it is what a marker expects. The `*` in `SELECT *` means "every column".

TEACH

COLUMN TYPES AND THE PRIMARY KEY

In `CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT)`, each column is a name then a type:

- ▶ `INTEGER` whole numbers, `TEXT` for text, `REAL` for decimals.
- ▶ **PRIMARY KEY** marks the column that identifies a row uniquely. Declared as `INTEGER PRIMARY KEY`, SQLite fills it in for you, counting up, so you never supply an `id` yourself.

That is why the example inserts only the name: `INSERT INTO students (name) VALUES ('Ada')` names the one column it is supplying, and `id` appears by itself.

The five steps

● MUST MEMORISE

EVERY DATABASE PROGRAM

Every program in this unit is the same five steps:

```
import sqlite3

conn = sqlite3.connect("school.db")    # 1 connect (creates the file if absent)
cursor = conn.cursor()                 # 2 get a cursor
cursor.execute("...")                  # 3 run SQL
conn.commit()                          # 4 save the changes
conn.close()                           # 5 disconnect
```

What each one is, since the question may ask:

- ▶ **connect** opens the database file, and **creates it if it does not exist**. It never fails for a missing file, unlike `open` in read mode.
- ▶ **A cursor** is the object you actually run SQL through and read results back from. Think of it as the hand that reaches into the database.
- ▶ **execute** runs one SQL statement, given as a string.
- ▶ **commit** makes your changes permanent. **Without it, your inserts are thrown away when the program ends**. No error, no warning, no data.
- ▶ **close** ends the connection.

Meant to create a table named students. Find the errors:

```
import sqlite3
conn = sqlite3.connect("school.db")
cursor = conn.cursor
cursor.execute("CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT)")
conn.commit
conn.close()
```

Two errors, and both are missing brackets. Exactly the `file.close` bug from Module Four, wearing a different hat.

- ▶ `conn.cursor` without `()` does not call the method. It hands you the method itself, so `cursor` is not a cursor at all. The next line then fails.
- ▶ `conn.commit` without `()` does nothing at all, silently. The table is never saved.

Only the first one speaks up:

```
Traceback (most recent call last):
  File "C:\Users\you\db.py", line 4, in <module>
    cursor.execute("CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT)")
AttributeError: 'builtin_function_or_method' object has no attribute 'execute'
```

Read that message closely, because it names the bug precisely: the thing you called `cursor` is a *method*, not a cursor, and methods have no `execute`. Corrected:

```
import sqlite3
conn = sqlite3.connect("school.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE students (id INTEGER PRIMARY KEY, name TEXT)")
conn.commit()
conn.close()
```

Watch out: run that corrected version **twice** and the second run fails, because the table already exists. See the next box.

● MUST MEMORISE

IF NOT EXISTS

Creating a table that is already there is an error:

```
sqlite3.OperationalError: table students already exists
```

Since `connect` keeps the file between runs, this bites the second time you run any program that creates a table. The fix is to say so in the SQL:

```
fields = "(id INTEGER PRIMARY KEY, name TEXT)"
cursor.execute("CREATE TABLE IF NOT EXISTS students " + fields)
```

Now it creates the table the first time and quietly does nothing afterwards. The examination asks for a table "if it doesn't already exist" in as many words, so **IF NOT EXISTS is the phrase that earns the mark.**

A question shows this and asks three things: explain it, say what happens when it runs, and state what must be true for it to succeed.

```
import sqlite3
conn = sqlite3.connect("school.db")
cursor = conn.cursor()
cursor.execute("INSERT INTO students (name) VALUES ('Ada')")
conn.commit()
conn.close()
```

What it does: connects to `school.db`, creating the file if absent; gets a cursor; adds one row to the `students` table with the name `Ada`, letting `id` fill itself; saves the change; closes.

What happens when it runs: nothing appears on screen. It is not a `SELECT`, so there is nothing to display. If it works you see no output at all, which is the correct answer and an easy mark to miss.

What must be true for it to succeed:

- ▶ The `students` table **must already exist**, with a `name` column. This code never creates it, so if the table is absent you get `sqlite3.OperationalError: no such table: students`.
- ▶ `id` must fill itself, i.e. be an `INTEGER PRIMARY KEY`, since no `id` is supplied.
- ▶ The file must be writable.

The single quotes around `'Ada'` are **SQL's** quotes, inside Python's double quotes. Two languages, one line.

Reading rows back

● MUST MEMORISE

SELECT AND FETCHALL

`execute` runs the `SELECT`, but it does not hand you the rows. You fetch them from the cursor:

```
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, 'Ada')
(2, 'Bola')
```

Each row comes back as a tuple, in the column order of the table. So `row[0]` is the `id` and `row[1]` is the name, and the round brackets in that output are Python showing you tuples, from Module Three, Unit 3.

`fetchall()` gives every row as a list of tuples. `fetchone()` gives just the next row, or `None` when there are none left.

A `SELECT` needs no `commit`, because it changes nothing.

Everything so far put fixed text in the SQL. Real data comes from variables, and there is a right way and a wrong way.

Right: a question mark for each value, and a tuple of the values.

```
cursor.execute("INSERT INTO contacts (name, phone) VALUES (?, ?)", (name, phone))
```

Wrong: building the SQL by joining strings.

```
sql = "INSERT INTO contacts (name, phone) VALUES ('" + name + "', '" + phone + "')"
cursor.execute(sql)
```

The joined version breaks the moment a name contains an apostrophe, which ends the SQL string early and leaves nonsense behind. A name like O'Brien is enough to do it. It is also how databases get attacked, by typing SQL into a name box.

The placeholder is a question mark, not a format specifier. Do not use an f-string here. And note the comma inside `(name,)` if you have only one value: without it, it is not a tuple.

A contact manager. Create `contacts.db` with a table `contacts` (`id`, `name`, `phone`), read three contacts from the user, insert each, then commit and close.

- 1 Connect and get a cursor.
connect creates `contacts.db` if it is absent
- 2 Create the table **if it does not already exist**. The question says those words, so use `IF NOT EXISTS`.
- 3 Loop three times with `range(3)`, reading `name` and `phone`.
phone stays `TEXT`: numbers can start with 0
- 4 Insert with placeholders, then commit once at the end and close.

CONTACTS.PY

```
import sqlite3

conn = sqlite3.connect("contacts.db")
cursor = conn.cursor()

cursor.execute("""CREATE TABLE IF NOT EXISTS contacts
                  (id INTEGER PRIMARY KEY, name TEXT, phone TEXT)""")

for i in range(3):
    name = input("Name: ")
    phone = input("Phone: ")
    cursor.execute("INSERT INTO contacts (name, phone) VALUES (?, ?)", (name, phone))

conn.commit()
conn.close()
print("Contacts saved.")
```

Entering Ada 08011112222, Bola 08033334444, John 08055556666:

Contacts saved.

The triple quoted string `"""..."""` lets a long piece of SQL run across two lines without fuss. It is an ordinary string, just one that may contain newlines.

Why phone is TEXT and not INTEGER: a phone number starting with 0 would lose that 0 as a number, and nobody ever adds two phone numbers together. Storing it as text is the correct choice, and saying so earns the mark.

The extension: accept contacts until the user types 'done' as the name, then fetch and display everything in the table. Sentinel loop from Module Three, plus SELECT .

CONTACTS_ANY.PY

```
import sqlite3

conn = sqlite3.connect("contacts.db")
cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS contacts
                  (id INTEGER PRIMARY KEY, name TEXT, phone TEXT)""")

while True:
    name = input("Name (or 'done'): ")
    if name == "done":
        break
    phone = input("Phone: ")
    cursor.execute("INSERT INTO contacts (name, phone) VALUES (?, ?)", (name, phone))

conn.commit()

cursor.execute("SELECT * FROM contacts")
for row in cursor.fetchall():
    print(f"{row[0]} | {row[1]} | {row[2]}")

conn.close()
```

Entering Ada 08011112222, then Bola 08033334444, then done:

```
1 | Ada | 08011112222
2 | Bola | 08033334444
```

Commit before you select. It happens to work without it here, but the habit is what keeps the marks, and the question asks for the changes to be saved.

Now you try: run it a second time and add John. What ids do you see? (1, 2 and 3. The file persists, so the old rows are still there and the new one continues the count. If you expected to see only John, you have just learned what "it lasts" means.)

1. What is a database, and give two reasons to use one?
2. What role does `sqlite3` play?
3. Name three SQL commands and their functions.
4. What happens if you forget `conn.commit()` ?
5. What does a row from `fetchall()` come back as?

ANSWERS

1. An organised collection of data held in tables of rows and columns, stored so it can be searched and changed efficiently. Any two of: it lasts beyond the program, it can be searched without reading everything, it enforces structure, it serves many users, it keeps data consistent.
2. It is the built in Python module that connects your program to an SQLite database file, letting you create tables, insert rows and query them from Python.
3. `CREATE TABLE` makes a table. `INSERT INTO` adds a row. `SELECT` fetches rows back.
4. Your changes are silently discarded when the program ends. No error is reported.
5. A tuple, with the columns in table order.

MODULE FIVE • UNIT 2

Python GUI Programming

Putting a window, a box and a button on the front of a program.

By the end of this unit you can: say what a GUI is, build a window with a label, an entry and a button, and wire the button to a function.

TEACH

A **GUI** is a Graphical User Interface: windows, buttons and boxes, rather than typing at a prompt. The opposite is the **command line interface**, which is everything you have written so far.

tkinter is Python's built in GUI module. Like `sqlite3`, it ships with Python.

A GUI program is inside out compared with everything so far. A normal program runs top to bottom and ends. A GUI program builds its window, then **waits**, and things happen only when the user does something. That waiting is a loop, and you start it yourself.

● MUST MEMORISE

THE FOUR WIDGETS

WIDGET	IS	MADE WITH
Window	The window itself	<code>tk.Tk()</code>
Label	Text the user reads	<code>tk.Label(window, text="...")</code>
Entry	A box the user types in	<code>tk.Entry(window)</code>
Button	Something to click	<code>tk.Button(window, text="...", command=fn)</code>

Every widget must be **placed** before it appears. `.pack()` is the simplest way: it stacks widgets down the window in the order you pack them. Create it and forget to pack it, and it simply is not there.

● MUST MEMORISE

COMMAND TAKES THE NAME, NOT THE CALL

A button is told which function to run when clicked:

```
tk.Button(window, text="Go", command=convert)
```

command=convert, not **command=convert()**. With brackets you would *call* it now, at the moment the button is built, and hand its return value to `command`. The button would then do nothing when clicked, and the work would have happened once, before the window even appeared.

Same rule as `file.close` against `file.close()` in Module Four, and `conn.cursor` against `conn.cursor()` in this module. **Brackets mean "do it now". No brackets means "the thing itself"**. Here you want the thing itself, because the button will do it later.

A window that converts Celsius to Fahrenheit. The formula is $F = C \times 9/5 + 32$.

- 1 Import `tkinter`, conventionally shortened to `tk`, and make the window.
- 2 Define what the button will do. It reads the entry with `.get()`, which hands back a **string**, exactly like `input`. Convert it.
`float(entry.get())`
- 3 Show the answer by changing the label's text with `.config(text=...)`.
- 4 Pack every widget, then start the wait with `mainloop()`.

CONVERTER.PY

```
import tkinter as tk

window = tk.Tk()
window.title("Celsius to Fahrenheit")

label = tk.Label(window, text="Celsius:")
entry = tk.Entry(window)
result = tk.Label(window, text="")

def convert():
    celsius = float(entry.get())
    fahrenheit = celsius * 9 / 5 + 32
    result.config(text=f"{fahrenheit:.1f} F")

button = tk.Button(window, text="Convert", command=convert)

label.pack()
entry.pack()
button.pack()
result.pack()

window.mainloop()
```

Type 100 and click Convert, and the bottom label reads:

212.0 F

`mainloop()` is the line people forget. Without it the program builds the window, reaches the end, and exits. The window flashes up and vanishes, or never appears at all. It must be the last line.

Now you try: what does 37 give? (98.6 F. Worth knowing: it is body temperature, so it is an easy check that your formula is the right way round.)

`entry.get()` hands back a string, always, exactly like `input()` in Module Two. Forget to convert and

```
celsius = entry.get()
fahrenheit = celsius * 9 / 5 + 32
```

fails with

```
TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

Read where that error actually lands, because it is instructive. `celsius * 9` **does not fail**: `"100" * 9` is a perfectly legal string repeated nine times, giving `"100100100100100100100100100100"`. It is the `/ 5` on the next step that has nowhere to go, because you cannot divide text.

So the error names the division, several characters after the mistake. **The line Python blames is not always the line you got wrong**, and here the real fault is the missing `float()` one line earlier.

It is the 53 trap from Module Two wearing a window.

1. What is a GUI, and what is the alternative called?
2. Name four tkinter widgets.
3. Why `command=convert` and not `command=convert()`?
4. What does `mainloop()` do, and what happens without it?
5. What type does `entry.get()` return?

ANSWERS

1. A Graphical User Interface: windows, buttons and boxes. The alternative is a command line interface.
2. `Window (Tk)`, `Label`, `Entry`, `Button`.
3. Without brackets you hand over the function itself, for the button to call when clicked. With brackets you call it immediately and hand over its result, so clicking does nothing.
4. It starts the loop that waits for the user. Without it the program ends at once and the window never appears.
5. A string, like `input()`. Convert it before doing arithmetic.

MOCK EXAMINATION ONE

Sit This Paper Under Exam Conditions

Same shape as the real thing, question for question: six questions, attempt any four, five parts each at three and a half marks. Different problems. Three hours, no computer, no notes. The answers start after the paper, and looking at them early is the only way to waste this.

Q1	(a) (b) (c) (d) (e)	17.5	
Q2	(a) (b) (c) (d) (e)	17.5	
Q3	(a) (b) (c) (d) (e)	17.5	
Q4	(a) (b) (c) (d) (e)	17.5	= 70 

Question Paper

● MUST MEMORISE

THE RUBRIC ITSELF

Course	CSC241, Python Programming Language I
--------	---------------------------------------

Credit unit	3
-------------	---

Time	3 hours
------	---------

Instruction	Answer Four (4) questions only
-------------	---------------------------------------

Marks	Six questions offered. Each is five parts at 3.5 marks, so 17.5 a question, 70 in total.
-------	--

Read the rubric before you write anything. Four questions, not six. Answering all six wastes an hour and earns nothing extra: only four are marked. Spend the first five minutes choosing your four, and pick them by part (d) and (e), which is where 7 of the 17.5 marks sit.

Question One

QUESTION ONE

17.5 MARKS

(a) Explain the differences among the three main types of control structures in Python. Give one example of each. **[3.5mks]**

(b) The following code is intended to print all odd numbers from 1 to 9, skipping the number 5. Identify and correct the error(s): **[3.5mks]**

```
n = 1
while n <= 9:
    if n % 2 != 0:
        if n == 5:
            continue
        print(n)
    n += 1
```

(c) Explain what the following code snippet does. What is the output? **[3.5mks]**

```
for i in range(1, 6):
    if i % 4 == 0:
        break
    print(i)
else:
    print("All done")
```

(d) You are writing the ticketing system for a cinema. The price depends on the customer's age. Your program should use decision making structures to implement the logic. **[3.5mks]**

Program requirements:

1. Prompt the user to enter an integer for the age.
2. Check that the age is positive.
3. If the age is under 13, print "Child - N500".
4. Else if the age is under 18, print "Teen - N800".
5. Else if the age is 60 or over, print "Senior - N600".
6. Else, print "Adult - N1200".
7. If the age is not positive, print "Please enter a valid age".

(e) Develop a new ticketing system as an extension of the one you did in (d) above. This updated version should allow the user to enter multiple ages in one input, separated by spaces. **[3.5mks]**

Program requirements:

1. Prompt the user to enter multiple ages separated by spaces.
2. Split the input and convert each item to an integer.
3. For each age entered, check it is positive, apply the pricing logic, and display the category.
4. After all ages, display the total number of tickets priced.

Question Two

QUESTION TWO

17.5 MARKS

(a) List and briefly describe four fundamental built-in data structures in Python. For each one, mention a real-world example where it can be appropriately used. **[3.5mks]**

(b) The following code is supposed to store the courses a student is taking in a list, remove any duplicates, and print the number of distinct courses. Identify and fix all the issues: **[3.5mks]**

```
courses = ["CSC241", "MAT121", "CSC241", "PHY121", "MAT121"]
courses = list(courses)
distinct = courses
print("Distinct courses:", len(distinct))
```

(c) What will be the output of the following code? Explain what happens at each stage of execution: **[3.5mks]**

```
student = {
    "name": "Bola",
    "level": 200,
    "courses": ["CSC241", "MAT121"]
}
student["courses"].append("PHY121")
student["level"] = 300
print(student["courses"])
print(len(student))
```

(d) You are building a stock checking tool for a shop. The program should allow the user to enter the quantities of four products, store them in a list, sort them in ascending order, and display the sorted quantities, the highest and the lowest. **[3.5mks]**

Program requirements:

1. Initialize an empty list to hold the quantities.
2. Use a loop to input the quantities of four products (as integers).
3. Store each quantity in the list.
4. Sort the list in ascending order.
5. Print the sorted list.
6. Print the maximum and minimum using appropriate built-in functions.

(e) Develop a new stock checking tool as an extension of (d). This version should allow any number of quantities (not just four), separated by spaces. **[3.5mks]**

Program requirements:

1. Prompt the user to enter all quantities, separated by spaces.
2. Convert the input to a list of integers.
3. Sort the quantities in descending order.
4. Display the sorted list, the highest and lowest, the average, and the number of products that need reordering. **A product needs reordering if its quantity is below 10.**

Question Three

QUESTION THREE

17.5 MARKS

(a) What is a function in Python? Why are functions important in programming? Differentiate between built-in functions and user-defined functions. Explain the two types of user-defined functions in Python. **[3.5mks]**

(b) The following code is meant to define a function that returns the cube of a number and then calls it. However, it does not work as expected. Identify and fix the errors: **[3.5mks]**

```
def cube(n)
    return n ** 3
print(Cube(3))
```

(c) Consider the following Python code. What will be the output? Explain how the function works, especially with regard to default parameters. **[3.5mks]**

```
def total(price, tax=0.1):
    return price + price * tax

print(total(100))
print(total(100, 0.2))
```

(d) You have been asked to build a results tool for a department. The program should define a function `grade(score)` that returns "A" for 70 and above, "B" for 60 to 69, "C" for 50 to 59, and "F" below 50. The program should prompt for the names and scores of three students, call the function for each, and display the result in the format: **[3.5mks]**

```
Student: Ada | Score: 72 | Grade: A
Student: Bola | Score: 45 | Grade: F
...
```

Program requirements:

1. Define `grade(score)` returning the letter as described.
2. In the main program, use a loop to collect each student's name and score.
3. For each student, call the function and determine the grade.
4. Display the result in a clean, formatted output.

(e) Develop a new results tool as an extension of (d). This version should allow any number of students (not just three), all in one session. **[3.5mks]**

Program requirements:

1. Define `grade(score)` as before.
2. Use a loop to repeatedly accept student names and scores.
3. Stop the loop when the user enters 'done' as the name.
4. For each student, call the function and display the grade.
5. After the loop, display summary statistics: how many students, and how many passed. **A pass is 50 and above.**

Question Four

QUESTION FOUR

17.5 MARKS

(a) What is a string in Python? List and briefly explain four common string methods used in Python. Provide an example for each method. **[3.5mks]**

(b) The following code is intended to extract the surname from a full name and convert it to lowercase. However, it contains errors. Identify and fix them: **[3.5mks]**

```
full_name = "Grace Hopper"
surname = full_name[full_name.index():]
print("Surname:", surname.lowercase())
```

(c) What will be the output of the following code? Explain the result of each line: **[3.5mks]**

```
tag = "   CSC241 Python   "
print(tag.strip())
print(tag.upper().replace("PYTHON", "CODE"))
print(len(tag.strip()))
```

(d) Write a Python program for a tool that processes a list of student registration numbers and prints the entry year. The registration numbers have the form YY/DEPT/NNN , and the entry year is the part before the first slash. **[3.5mks]**

Program Requirements:

1. Create a list containing at least five registration numbers.
2. Loop through the list.
3. For each one, split the string at the slash and extract the year.
4. Print the result in a clean format, such as:

```
Reg: 24/CSC/001 | Year: 24
```

(e) Develop a new text tool as an extension of (d). This version should help a user analyse multiple sentences entered in a single paragraph. **[3.5mks]**

Program Requirements:

1. Prompt the user to enter a paragraph of text.
2. Split the paragraph into sentences using "." as the delimiter.
3. For each sentence: remove leading and trailing whitespace, skip empty sentences, count the number of words, convert to lowercase, and count the number of consonants (letters that are not a, e, i, o, u).
4. Display the results for that sentence.

Question Five

QUESTION FIVE

17.5 MARKS

(a) What is file handling in Python? List and explain four file access modes used in the `open()` function. Also mention the difference between reading from a file and appending to a file. **[3.5mks]**

(b) The following code is intended to write a heading to a file named `report.txt`, but it contains errors. Identify and fix them: **[3.5mks]**

```
f = open("report.txt", "w")
f.write("Weekly Report")
f.close
```

(c) What will be the result of executing the following code? Explain what each line does. **[3.5mks]**

```
with open("log.txt", "a") as f:
    f.write("Entry recorded.\n")

print("Log updated.")
```

(d) Develop a Python program for a library book log. The program should allow the librarian to enter the titles of books borrowed today and save the list to a file called `borrowed.txt`. **[3.5mks]**

Program Requirement:

1. Ask the librarian how many books were borrowed.
2. Use a loop to input each title.
3. Open `borrowed.txt` in write mode.
4. Write each title to the file on a new line.
5. After saving, print "Log saved successfully."

(e) Develop a new library record system as an extension of (d). This version should allow the user to enter multiple records (title and days overdue) until 'done' is entered as the title. Each entry should be stored in a text file called `fines.txt` in the format:

[3.5mks]

```
Things Fall Apart, 3
Purple Hibiscus, 0
```

Program Requirement:

1. Open `fines.txt` in write mode.
2. Use a loop to collect titles and days overdue until 'done' is entered.
3. Write each entry in the format: Title, Days.
4. Close the file, then reopen it in read mode.
5. For each line: split the title and days, print the record, and total the fine. **The fine is N50 per day overdue.**
6. Display the total fine owed.

Question Six

QUESTION SIX

17.5 MARKS

(a) i. What is a database, and why is it used in programming? ii. What role does the `sqlite3` module play in Python? iii. Mention three common SQL commands used when working with databases and their functions. **[3.5mks]**

(b) The following code is meant to create a database table named `books`, but it contains errors. Identify and fix the issues: **[3.5mks]**

```
import sqlite3
conn = sqlite3.connect("library.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE books (id INTEGER PRIMARY KEY, title TEXT)")
conn.commit()
conn.close()
```

(c) i. Examine the following code and explain what it does. ii. What will happen when this code runs? iii. What assumptions must be true for it to execute successfully? **[3.5mks]**

```
import sqlite3
conn = sqlite3.connect("library.db")
cursor = conn.cursor()
cursor.execute("INSERT INTO books (title) VALUES ('Things Fall Apart')")
conn.commit()
conn.close()
```

(d) Develop a simple Python-based student registry using `sqlite3`. The program should create a table `students` with fields: `id`, `name` and `matric`. It prompts the user to enter the name and matric number of three students and inserts each into the database. **[3.5mks]**

Program requirements:

1. Import `sqlite3` and connect to a database named `registry.db`.
2. Create a table `students` if it does not already exist.
3. Use a loop to collect name and matric number for three students.
4. Insert each student into the table.
5. Commit the changes and close the connection.

(e) Develop a new student registry as an extension of (d). This version should allow the user to enter multiple students until they type 'done' as the name, store the records in `registry.db`, and after all entries are added, fetch and display all students stored in the table. **[3.5mks]**

The answers follow. **Do not read them until you have written yours.**

Reading a worked answer produces a strong, false feeling of competence: everything looks obvious once somebody else has done it. The only way to find out whether you can write a sentinel loop from nothing is to sit with a blank page and fail to, which is unpleasant and is the entire point. Three hours now is worth more than three days of rereading.

Answers

Every output below was produced by running the code, not by reading it. Where your answer differs in wording but not in substance, you are right.

Question One

ANSWER

QUESTION ONE (A)

THREE CONTROL STRUCTURES, ONE EXAMPLE EACH

- ▶ **Sequence.** Statements run one after another, top to bottom, with no choices.
Example: `a = 2` then `b = a + 1`.
- ▶ **Selection** (decision). A block runs only if a condition is true. Written with `if`, `elif`, `else`. Example: `if n % 2 == 0: print("even")`.
- ▶ **Iteration** (repetition). A block runs many times. Written with `while` or `for`.
Example: `for i in range(3): print(i)`.

The question says "give one example of each", so a sentence without code will not score full marks. Write the three lines.

ANSWER

QUESTION ONE (B)

THE ERROR

The code **does not just print the wrong numbers. It never stops.** It prints 1, then 3, and then hangs forever.

When `n` reaches 5, `continue` jumps straight back to the top of the `while`, **skipping** `n += 1` on the way. So `n` stays 5 for ever and the condition `n <= 9` never turns false.

THE FIX

```
n = 1
while n <= 9:
    if n % 2 != 0 and n != 5:
        print(n)
    n += 1
```

```
1
3
7
9
```

Removing `continue` removes the bug at its root. If you keep `continue`, `n += 1` must happen before it, or the loop hangs again.

Say the word "infinite" in your answer. A candidate who writes "it prints 5 by mistake" has not spotted the actual error and will not get the marks.

OUTPUT

```
1
2
3
```

EXPLANATION

The loop counts 1, 2, 3, 4, 5. It prints each until `i` is 4, where `4 % 4 == 0` is true and `break` leaves the loop at once, before the `print`. So 4 and 5 never print.

"All done" never prints. A loop's `else` runs only when the loop finishes *normally*. A `break` skips it. Read `else` here as "if no break happened".

This is the marked difference between a good answer and a lucky one. Most candidates add "All done" to the output.

```
age = int(input("Enter age: "))

if age > 0:
    if age < 13:
        print("Child - N500")
    elif age < 18:
        print("Teen - N800")
    elif age >= 60:
        print("Senior - N600")
    else:
        print("Adult - N1200")
else:
    print("Please enter a valid age")
```

Run with 8, 15, 70, 30 and 0:

```
Child - N500
Teen - N800
Senior - N600
Adult - N1200
Please enter a valid age
```

WHY THE ORDER IS WHAT IT IS

The tests must run narrowest first. `age < 13` before `age < 18`, because a 10 year old satisfies both and the first true test wins. Swap them and no child could ever be priced.

`elif age >= 60` is reached only by people 18 or over, because anything younger was caught above. That is why it needs no upper bound.

```
line = input("Enter ages separated by spaces: ")
count = 0

for part in line.split():
    age = int(part)
    if age > 0:
        if age < 13:
            print(age, "Child - N500")
        elif age < 18:
            print(age, "Teen - N800")
        elif age >= 60:
            print(age, "Senior - N600")
        else:
            print(age, "Adult - N1200")
        count += 1
    else:
        print(age, "Please enter a valid age")

print("Tickets priced:", count)
```

Entering 8 15 70 30 :

```
Enter ages separated by spaces: 8 15 70 30
8 Child - N500
15 Teen - N800
70 Senior - N600
30 Adult - N1200
Tickets priced: 4
```

WHAT EARNED THE MARKS

The decision logic is **identical** to part (d). Only two things changed: `split()` replaced the single `input`, and a `for` loop wrapped the whole thing. That is what part (e) is testing, every single year.

`count` is incremented inside the positive branch only, because an invalid age is not a ticket.

Question Two

ANSWER

QUESTION TWO (A)

STRUCTURE	DESCRIPTION	REAL WORLD EXAMPLE
List []	Ordered, changeable, allows duplicates	The register of a class, in order
Tuple ()	Ordered, cannot be changed	The latitude and longitude of a place
Set { }	Unordered, no duplicates	The distinct courses a student is taking
Dictionary {k: v}	Pairs looked up by key	A student's matric number against their name

The question asks for a real-world example for each, so a bare definition scores half. Give the example.

ANSWER

QUESTION TWO (B)

THE ISSUES

The code runs perfectly and prints the wrong answer. There is no error message to find, which is the difficulty.

```
Distinct courses: 5
```

- ▶ `list(courses)` copies a list that is already a list. It does nothing useful and removes no duplicates.
- ▶ `distinct = courses` merely gives the same list a second name. Nothing was deduplicated at all.

THE FIX

```
courses = ["CSC241", "MAT121", "CSC241", "PHY121", "MAT121"]
distinct = set(courses)
print("Distinct courses:", len(distinct))
```

```
Distinct courses: 3
```

The distinct courses are CSC241, MAT121 and PHY121. The question allows a `for` loop instead of a `set`, which is worth the same marks but more writing:

```
distinct = []
for c in courses:
    if c not in distinct:
        distinct.append(c)
```

OUTPUT

```
['CSC241', 'MAT121', 'PHY121']  
3
```

EACH STAGE

- ▶ `student["courses"]` looks up the key and returns **the list itself**, not a copy. `.append("PHY121")` then changes that list, so the dictionary's value is changed. No reassignment was needed.
- ▶ `student["level"] = 300` replaces an existing value. The key already existed, so nothing is added.
- ▶ `print(student["courses"])` shows the three courses.
- ▶ `len(student)` is **3**, because **len on a dictionary counts its keys**: name, level and courses. It does not count the courses inside it, and it did not change when level was reassigned.

The trap is the last line. Answering 4 means you counted something else. There were three keys at the start and three at the end.

```
quantities = []  
  
for i in range(4):  
    q = int(input("Enter quantity: "))  
    quantities.append(q)  
  
quantities.sort()  
print("Sorted:", quantities)  
print("Highest:", max(quantities))  
print("Lowest:", min(quantities))
```

Entering 25, 4, 60, 12:

```
Sorted: [4, 12, 25, 60]  
Highest: 60  
Lowest: 4
```

TWO WAYS TO LOSE MARKS HERE

- ▶ `quantities = quantities.sort()` would leave `None` in the variable and everything after would fail. `sort()` sorts in place and returns nothing. Call it on its own line.
- ▶ Forgetting `int()` means you sort strings, so `"12"` would come before `"4"` alphabetically.

```
line = input("Enter quantities separated by spaces: ")

quantities = []
for part in line.split():
    quantities.append(int(part))

quantities.sort(reverse=True)

reorder = 0
for q in quantities:
    if q < 10:
        reorder += 1

average = sum(quantities) / len(quantities)

print("Sorted:", quantities)
print("Highest:", max(quantities))
print("Lowest:", min(quantities))
print(f"Average: {average:.2f}")
print("Need reordering:", reorder)
```

Entering 25 4 60 12 8:

```
Enter quantities separated by spaces: 25 4 60 12 8
Sorted: [60, 25, 12, 8, 4]
Highest: 60
Lowest: 4
Average: 21.80
Need reordering: 2
```

THE DETAILS THAT EARN THE MARKS

- ▶ `sort(reverse=True)` for descending. The question says descending.
- ▶ `{average:.2f}`. The average is a float and will often be a long ugly one. Format it. This is the difference between a professional answer and a puzzling one.
- ▶ `q < 10`, strictly below, because the question says "below 10". A quantity of exactly 10 does **not** need reordering. Here 4 and 8 do.

Question Three

ANSWER

QUESTION THREE (A)

What a function is: a named block of code that runs only when it is called. It may take values in and may hand a value back.

Why they matter: reuse (write once, call anywhere); readability (a name says what a block does); maintenance (fix a bug in one place); decomposition (a large problem splits into small testable pieces).

Built in against user defined: a built in function comes with Python and is always available (`print` , `len` , `int` , `max`). A user defined function is one you write with `def` , and it must be defined before it is called.

The two types of user defined function:

- ▶ **Void:** does something and returns nothing. It has no `return` , so calling it gives `None` . Example: a function that only prints.
- ▶ **Value returning:** works something out and hands it back with `return` , so the call can be used as a value.

Four things are asked here and each is worth roughly a quarter. Answer all four explicitly, with a heading for each if it helps you not to forget one.

ANSWER

QUESTION THREE (B)

THE ERRORS

- ▶ **No colon** after `def cube(n)` . This is a `SyntaxError`, found while reading the file, so **nothing runs at all**.
- ▶ **Cube with a capital C** does not exist. The function is `cube` , and names are case sensitive.

Only the first is reported. Fix it, run again, and only then do you see:

```
NameError: name 'Cube' is not defined. Did you mean: 'cube'?
```

THE FIX

```
def cube(n):  
    return n ** 3  
  
print(cube(3))
```

```
27
```

Say **both** errors. The question says "errors", plural, and the marks are split. Say also that the first hides the second: it shows you understand why a `SyntaxError` stops everything.

ANSWER

QUESTION THREE (C)

OUTPUT

```
110.0
120.0
```

EXPLANATION

- ▶ `tax=0.1` is a **default parameter**: a value used when the caller does not supply one.
- ▶ `total(100)` passes only `price`, so `tax` falls back to `0.1`. The result is $100 + 10.0 = 110.0$.
- ▶ `total(100, 0.2)` supplies `tax`, and the argument **overrides** the default. The result is $100 + 20.0 = 120.0$.

Both answers are floats, printed as `110.0` and `120.0`, not `110` and `120`. The multiplication by a float makes the whole expression a float. Writing `110` without the decimal point is not the output.

Worth stating in your answer: a parameter with a default must come after any without one, so `def total(tax=0.1, price)` would be a `SyntaxError`.

ANSWER

QUESTION THREE (D)

```
def grade(score):
    if score >= 70:
        return "A"
    elif score >= 60:
        return "B"
    elif score >= 50:
        return "C"
    else:
        return "F"

for i in range(3):
    name = input("Student name: ")
    score = int(input("Score: "))
    print(f"Student: {name} | Score: {score} | Grade: {grade(score)}")
```

Entering Ada 72, Bola 45, John 65:

```
Student: Ada | Score: 72 | Grade: A
Student: Bola | Score: 45 | Grade: F
Student: John | Score: 65 | Grade: B
```

TWO THINGS THE MARKER IS LOOKING FOR

- ▶ **return, not print**. The question says the function "returns" the letter. A function that prints instead hands back `None`, and the caller has nothing to display.
- ▶ **Descending order of tests**. 70 first. Test 50 first and every score above 50 would be a C.

`grade(score)` is called **inside the f-string**, which is neat and legal. Assigning it to a variable first is equally correct.

```
def grade(score):
    if score >= 70:
        return "A"
    elif score >= 60:
        return "B"
    elif score >= 50:
        return "C"
    else:
        return "F"

total = 0
passed = 0

while True:
    name = input("Student name (or 'done'): ")
    if name == "done":
        break
    score = int(input("Score: "))
    print(f"Student: {name} | Score: {score} | Grade: {grade(score)}")
    total += 1
    if score >= 50:
        passed += 1

print(f"Students: {total}, Passed: {passed}, Failed: {total - passed}")
```

Entering Ada 72, Bola 45, John 65, then done:

```
Student: Ada | Score: 72 | Grade: A
Student: Bola | Score: 45 | Grade: F
Student: John | Score: 65 | Grade: B
Students: 3, Passed: 2, Failed: 1
```

THE SHAPE

The function is untouched. Only the loop changed, from `for i in range(3)` to a sentinel `while True` with a `break`. That is the whole of part (e), and it is why a function is worth writing in part (d) in the first place.

Test for 'done' before asking for the score, or the program will demand a score for a student who does not exist.

A pass is 50 and above, so the test is `>=`. Bola's 45 fails.

Question Four

ANSWER

QUESTION FOUR (A)

What a string is: a sequence of characters, written in single or double quotes. It is **immutable**, so it cannot be changed in place.

METHOD	EXPLANATION	EXAMPLE	GIVES
<code>upper()</code>	Returns a copy in capitals	<code>"ada".upper()</code>	<code>'ADA'</code>
<code>lower()</code>	Returns a copy in small letters	<code>"ADA".lower()</code>	<code>'ada'</code>
<code>strip()</code>	Returns a copy with whitespace removed from both ends	<code>" hi ".strip()</code>	<code>'hi'</code>
<code>replace(a, b)</code>	Returns a copy with every a changed to b	<code>"a-b".replace("-", " ")</code>	<code>'a b'</code>

Notice every explanation says **"returns a copy"**. That single word shows the marker you understand immutability, and it costs you nothing to write.

ANSWER

QUESTION FOUR (B)

THE ERRORS

- ▶ `full_name.index()` is called with **nothing to search for**. It needs the text whose position you want, here the space. `TypeError: index expected at least 1 argument, got 0`
- ▶ `lowercase()` **does not exist**. The method is `lower()`.

THE SUBTLETY THAT COSTS HALF THE MARKS

Fixing `index()` to `index(" ")` is not enough. That gives the position **of the space**, which is 5, and slicing from 5 **includes the space**:

```
>>> full_name = "Grace Hopper"
>>> full_name[full_name.index(" "):]
' Hopper'
```

Note the leading space inside the quotes. You must start **one past** it:

THE FIX

```
full_name = "Grace Hopper"
surname = full_name[full_name.index(" ") + 1:]
print("Surname:", surname.lower())
```

```
Surname: hopper
```

A cleaner answer, worth full marks and less error prone:

```
surname = full_name.split()[1]
print("Surname:", surname.lower())
```

OUTPUT

```
CSC241 Python
CSC241 CODE
13
```

EACH LINE

- ▶ `tag.strip()` removes the three spaces at each end and returns `CSC241 Python`. **tag itself is unchanged.**
- ▶ `tag.upper()` gives `" CSC241 PYTHON "`. **Now** `replace("PYTHON", "CODE")` matches, because the text is in capitals by the time `replace` sees it. **The spaces are still there**, because `strip` on the line before returned a new string and changed nothing.
- ▶ `len(tag.strip())` is **13**: six for `CSC241`, one space, six for `Python`. Note it is `len` of the *stripped* string, not of `tag`, which is 19.

Two traps in three lines. The middle line only matches because of the `upper()` before it, and the last line only gives 13 because of the `strip()` inside it.

```
regs = [
    "24/CSC/001",
    "23/CSC/117",
    "24/MAT/042",
    "22/PHY/008",
    "24/CSC/236",
]

for reg in regs:
    year = reg.split("/")[0]
    print(f"Reg: {reg} | Year: {year}")
```

```
Reg: 24/CSC/001 | Year: 24
Reg: 23/CSC/117 | Year: 23
Reg: 24/MAT/042 | Year: 24
Reg: 22/PHY/008 | Year: 22
Reg: 24/CSC/236 | Year: 24
```

WHY INDEX 0

`"24/CSC/001".split("/")` gives `['24', 'CSC', '001']`. The year is the part before the first slash, so it is item **0**. The separator itself is thrown away.

The year stays a **string**, which is correct here: it is a label, not a number, and `"001"` would lose its zeros as an integer.

```
paragraph = input("Enter a paragraph: ")

for sentence in paragraph.split("."):
    sentence = sentence.strip()
    if sentence == "":
        continue

    words = len(sentence.split())

    consonants = 0
    for ch in sentence.lower():
        if ch.isalpha() and ch not in "aeiou":
            consonants += 1

    print(f"Sentence: {sentence.lower()}")
    print(f" Words: {words}, Consonants: {consonants}")
```

Entering Python is fun. I like code.

```
Enter a paragraph: Python is fun. I like code.
Sentence: python is fun
Words: 3, Consonants: 8
Sentence: i like code
Words: 3, Consonants: 4
```

THE ONE THAT CATCHES EVERYBODY

ch.isalpha() is not optional. A consonant is a *letter* that is not a vowel. Without that test the **spaces** get counted, since a space is certainly not in "aeiou", and the answer comes out as 10 instead of 8.

The eight consonants in "python is fun" are p, y, t, h and n from *python*, s from *is*, then f and n from *fun*. The vowels o, i and u are excluded, and the two spaces are excluded by `isalpha()`.

y counts as a consonant here, because the question defines vowels as a, e, i, o, u and y is not among them. Follow the question's definition, not your instinct about English.

The `continue` is needed because the paragraph ends in a full stop, so splitting leaves an empty final piece.

Question Five

ANSWER

QUESTION FIVE (A)

File handling is working with files on disk from a program, so that data survives after the program ends. It is always three steps: open, process, close.

MODE	NAME	BEHAVIOUR
"r"	Read	Opens for reading. Error if the file does not exist.
"w"	Write	Erases the file completely , then writes. Creates it if absent.
"a"	Append	Adds to the end, keeping what is there. Creates it if absent.
"x"	Exclusive create	Creates it. Error if it already exists.

Reading against appending: reading takes data *out* of the file and changes nothing, and the file must already exist. Appending puts data *into* the file at the end, keeps everything already there, and creates the file if needed.

Worth adding, and it shows real understanding: the dangerous distinction is "w" against "a". Opening with "w" destroys the contents immediately, before you write anything.

ANSWER

QUESTION FIVE (B)

THE ERRORS

- ▶ The write line is **missing its closing bracket**. This is a `SyntaxError`, so **nothing in the file runs**.
- ▶ `f.close` has **no brackets**, so it is not a call. It names the method and discards it. The file is never closed.

The first hides the second, and the second is the more dangerous, because it reports nothing at all: no error, no message, and possibly an empty file.

THE FIX

```
f = open("report.txt", "w")
f.write("Weekly Report")
f.close()
```

Better still, and worth saying:

```
with open("report.txt", "w") as f:
    f.write("Weekly Report")
```

`with` closes the file for you, so the second bug becomes impossible.

RESULT

```
Log updated.
```

EACH LINE

- ▶ `with open("log.txt", "a") as f` opens `log.txt` in **append** mode, **creating it if it does not exist**, and calls it `f`.
- ▶ `f.write("Entry recorded.\n")` adds that line **to the end** of the file, keeping everything already in it. The `\n` is needed because `write` adds no newline of its own.
- ▶ The block ends, so `with` **closes the file automatically**. There is no `close()` to forget.
- ▶ `print("Log updated.")` displays that text on screen.

Only "Log updated." appears on screen. The written line went into the file, not to the display. Saying so is the mark. Run it three times and the file has three entries, while the screen still shows one line each time.

```
count = int(input("How many books were borrowed? "))

with open("borrowed.txt", "w") as f:
    for i in range(count):
        title = input("Title: ")
        f.write(title + "\n")

print("Log saved successfully.")
```

Entering 2, then Things Fall Apart, then Purple Hibiscus:

```
Log saved successfully.
```

and `borrowed.txt` now holds:

```
Things Fall Apart
Purple Hibiscus
```

THE DETAIL

+ `"\n"` **is essential**. The requirement says "on a new line", and `write` adds none. Without it the file would hold `Things Fall ApartPurple Hibiscus` on one line. An `f`-string works equally well: `f.write(f"{title}\n")`.


```
with open("fines.txt", "w") as f:
    while True:
        title = input("Title (or 'done'): ")
        if title == "done":
            break
        days = int(input("Days overdue: "))
        f.write(f"{title}, {days}\n")

total_fine = 0
with open("fines.txt", "r") as f:
    for line in f:
        title, days = line.strip().split(", ")
        days = int(days)
        fine = days * 50
        total_fine += fine
        print(f"{title}: {days} days, N{fine}")

print("Total fine: N" + str(total_fine))
```

Entering Things Fall Apart 3, Purple Hibiscus 0, Half of a Yellow Sun 2, then done:

```
Things Fall Apart: 3 days, N150
Purple Hibiscus: 0 days, N0
Half of a Yellow Sun: 2 days, N100
Total fine: N250
```

THE DETAILS THAT DECIDE THIS

- ▶ **line.strip() before split(", ")**. Every line read from a file keeps its `\n`. Without the strip, `days` would be `"3\n"`, and `int("3\n")` happens to work, but the habit will bite you elsewhere. Strip first, always.
- ▶ **split(", ") with the space**, matching exactly the format you wrote. Splitting on `","` alone leaves a leading space on the number.
- ▶ The fine is N50 per day, so a book 0 days overdue owes N0 and still appears in the list. Do not skip it.

Question Six

ANSWER

QUESTION SIX (A)

i. What a database is, and why: an organised collection of data held in tables of rows and columns, stored so it can be searched, changed and retrieved efficiently. It is used because the data lasts beyond the program; it can be searched without reading everything; it enforces structure, so a column keeps its type; it serves many users at once; and it keeps data consistent through rules such as a primary key.

ii. The role of `sqlite3`: it is the built in Python module that connects a Python program to an SQLite database file, letting you create tables, insert rows and query them from Python. It ships with Python, so nothing needs installing.

iii. Three SQL commands:

COMMAND	FUNCTION
CREATE TABLE	Makes a new table and defines its columns
INSERT INTO	Adds a new row of data to a table
SELECT	Retrieves rows from a table

ANSWER

QUESTION SIX (B)

THE ISSUES

Both are missing brackets, and they behave very differently.

- ▶ `conn.cursor` does not call the method, so `cursor` holds the method itself rather than a cursor. The next line then fails with `AttributeError`:
'builtin_function_or_method' object has no attribute 'execute'.
- ▶ `conn.commit` does nothing at all, **silently**. The table would never be saved.

THE FIX

```
import sqlite3
conn = sqlite3.connect("library.db")
cursor = conn.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS books (id INTEGER PRIMARY KEY, title TEXT)")
conn.commit()
conn.close()
```

Worth the extra mark: add `IF NOT EXISTS`. Without it the fixed program works once and then fails on every later run with `sqlite3.OperationalError: table books already exists`, because `connect` keeps the file between runs.

i. What it does: connects to `library.db`, creating the file if it does not exist; gets a cursor; runs an `INSERT` adding one row to the `books` table with the title 'Things Fall Apart', letting `id` fill itself; commits, making the change permanent; and closes the connection.

ii. What happens when it runs: nothing is displayed at all. There is no `print` and no `SELECT`, so a successful run produces no output. One row is silently added to the table.

iii. The assumptions:

- ▶ The `books` table **must already exist**, with a `title` column. This code never creates it, so without it you get `sqlite3.OperationalError: no such table: books`.
- ▶ `id` must fill itself, that is, be declared `INTEGER PRIMARY KEY`, since no `id` is supplied.
- ▶ The file must be writable, and the disk must have room.

Part ii is the easy mark most candidates miss. "No output" is the answer, and it is worth saying in those words.

```
import sqlite3

conn = sqlite3.connect("registry.db")
cursor = conn.cursor()

cursor.execute("""CREATE TABLE IF NOT EXISTS students
                  (id INTEGER PRIMARY KEY, name TEXT, matric TEXT)""")

for i in range(3):
    name = input("Name: ")
    matric = input("Matric: ")
    cursor.execute("INSERT INTO students (name, matric) VALUES (?, ?)", (name, matric))

conn.commit()
conn.close()
print("Students saved.")
```

Entering Ada 24/CSC/001, Bola 24/CSC/117, John 24/MAT/042:

Students saved.

WHAT EARNS THE MARKS

- ▶ **IF NOT EXISTS**, because the question says "if it does not already exist" in as many words.
- ▶ **The ? placeholders**, not string joining. A matric number contains slashes and a name may contain an apostrophe, which would break joined SQL outright.
- ▶ **matric is TEXT**, not `INTEGER`. It contains slashes, so it is not a number at all.
- ▶ `commit()` before `close()`, or nothing is saved.

```
import sqlite3

conn = sqlite3.connect("registry.db")
cursor = conn.cursor()
cursor.execute("""CREATE TABLE IF NOT EXISTS students
                  (id INTEGER PRIMARY KEY, name TEXT, matric TEXT)""")

while True:
    name = input("Name (or 'done'): ")
    if name == "done":
        break
    matric = input("Matric: ")
    cursor.execute("INSERT INTO students (name, matric) VALUES (?, ?)", (name, matric))

conn.commit()

cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(f"{row[0]} | {row[1]} | {row[2]}")

conn.close()
```

Entering Ada 24/CSC/001, Bola 24/CSC/117, then done:

```
1 | Ada | 24/CSC/001
2 | Bola | 24/CSC/117
```

THE SHAPE, ONE LAST TIME

The sentinel loop is **exactly** the one from Questions Three and Five. The only new thing in this question is the `SELECT` and `fetchall()` at the end, which the question asks for in its last line.

Each row is a tuple, so `row[0]`, `row[1]` and `row[2]` are id, name and matric, in the order the table declares them.

Commit before you select.

● MUST MEMORISE

WHAT THIS PAPER WAS REALLY TESTING

Look back at the six questions you just answered. Underneath the different scenarios, **every part (e) was the same two lines of thinking:**

- ▶ **The sentinel loop:** `while True`, read, test for 'done', `break`, otherwise process. Questions Three, Five and Six.
- ▶ **Split and convert:** read one line, `split()`, loop, convert each with `int()`. Questions One and Two.

That is 17.5 marks of the 70 you can attempt, and it is two patterns. If you can write both from a blank page, without thinking, you have banked a quarter of the paper before you read a single question.

And **every part (b) was a hidden second error:** a `SyntaxError` masking a `NameError`, an unclosed bracket masking a missing pair of brackets. Look for two.

MOCK EXAMINATION TWO

Sit This One After the First

The same six questions, the same five parts, the same three hours. New problems throughout, and the parts that caught you in Mock One are asked here from the other side. Sit it cold, on paper, with the book shut.

Q1	(a) (b) (c) (d) (e)	17.5	answer	
Q2	(a) (b) (c) (d) (e)	17.5	answer	
Q3	(a) (b) (c) (d) (e)	17.5	answer	
Q4	(a) (b) (c) (d) (e)	17.5	answer	= 70 
Q5	(a) (b) (c) (d) (e)	17.5	leave it	
Q6	(a) (b) (c) (d) (e)	17.5	leave it	

Question Paper

● MUST MEMORISE

FOUR, NOT SIX

Course	CSC241, Python Programming Language I
--------	---------------------------------------

Credit unit	3
-------------	---

Time	3 hours
------	---------

Instruction	Answer Four (4) questions only
-------------	---------------------------------------

Marks	Six questions offered. Each is five parts at 3.5 marks, so 17.5 a question, 70 in total.
-------	--

You chose four questions in Mock One. Choose four different ones here. The rubric is generous in a way students miss: **two whole questions can be subjects you never mastered**, and you still leave with full marks. The skill is picking, and you pick on parts (d) and (e).

Question One

QUESTION ONE

17.5 MARKS

(a) What is a control structure? Explain `break`, `continue` and the `else` branch of a loop, and give one short example of each. **[3.5mks]**

(b) The following code is intended to add up the numbers 1 to 4 and print the total. Identify and correct the error(s): **[3.5mks]**

```
total = 0
for i in range(1, 5)
    total += i
print("Total:", Total)
```

(c) Explain what the following code does. What is the output, and why does the last line print at all? **[3.5mks]**

```
for i in range(3, 0, -1):
    print(i)
else:
    print("Liftoff")
```

(d) You are writing the billing program for an electricity company. The charge depends on the number of units consumed. Your program should use decision making structures to implement the logic. **[3.5mks]**

Program requirements:

1. Prompt the user to enter an integer for the units consumed.
2. Check that the reading is positive.
3. If the units are under 50, the band is "Lifeline" at N10 per unit.
4. Else if the units are under 200, the band is "Domestic" at N20 per unit.
5. Else, the band is "Commercial" at N30 per unit.
6. Print the band and the bill, which is the units times the rate.
7. If the reading is not positive, print "Please enter a valid reading".

(e) Develop a new billing program as an extension of the one you did in (d) above. This updated version should allow the user to enter several meter readings in one input, separated by spaces. **[3.5mks]**

Program requirements:

1. Prompt the user to enter all the readings separated by spaces.
2. Split the input and convert each item to an integer.
3. For each reading, check it is positive, apply the banding, and display the band and the bill.
4. After all readings, display how many were billed and the total of all the bills.

Question Two

QUESTION TWO

17.5 MARKS

(a) Differentiate between a list and a tuple, and say when each is the right choice. What is a set for, and what does a dictionary give you that a list cannot? Give a real-world example of each of the four. **[3.5mks]**

(b) The following code is supposed to add a parcel weight to the collection and print the weights in order. It fails, and fixing the first fault reveals a second. Identify and fix both: **[3.5mks]**

```
weights = (12, 5, 20, 8)
weights.append(15)
print("Sorted:", weights.sort())
```

(c) What will be the output of the following code? Explain what happens at each stage of execution: **[3.5mks]**

```
stock = {"pen": 12, "book": 5}
stock["pen"] = stock["pen"] - 3
stock["bag"] = 7
print(stock["pen"])
print(len(stock))
print("book" in stock)
```

(d) You are building a weighing tool for a courier. The program should allow the user to enter the weights of four parcels, store them in a list, sort them in ascending order, and display the sorted weights, the heaviest and the lightest. **[3.5mks]**

Program requirements:

1. Initialize an empty list to hold the weights.
2. Use a loop to input the weights of four parcels (as integers).
3. Store each weight in the list.
4. Sort the list in ascending order.
5. Print the sorted list.
6. Print the heaviest and the lightest using appropriate built-in functions.

(e) Develop a new weighing tool as an extension of (d). This version should accept any number of parcels, not just four, entered separated by spaces. **[3.5mks]**

Program requirements:

1. Prompt the user to enter all the weights, separated by spaces.
2. Convert the input to a list of integers.
3. Sort the weights in descending order.
4. Display the sorted list, the heaviest and lightest, the average weight, and the number of parcels that attract a surcharge. **A parcel attracts a surcharge if it is above 20 kg.**

Question Three

QUESTION THREE

17.5 MARKS

(a) What is the difference between a parameter and an argument? What does `return` do, and what does a function give back if you leave it out? What is a default parameter, and why must it come last? **[3.5mks]**

(b) The following code is meant to define a function that gives the parking fee for a number of hours and then print it. Correcting the first error exposes a second. Identify and fix both: **[3.5mks]**

```
def fee(hours)
    total = 200 + (hours - 1) * 150
print(fee(3))
```

(c) Consider the following Python code. What will be the output? Explain how the function works, especially with regard to default parameters. **[3.5mks]**

```
def pay(hours, rate=500):
    return hours * rate

print(pay(8))
print(pay(8, 700))
```

(d) You have been asked to build a car park till. The program should define a function `fee(hours)` that charges N200 for the first hour and N150 for each hour after it. The program should prompt for the plate number and hours of three vehicles, call the function for each, and display the result in the format: **[3.5mks]**

```
Plate: ABC123 | Hours: 3 | Fee: N500
Plate: XYZ789 | Hours: 1 | Fee: N200
...
```

Program requirements:

1. Define `fee(hours)` returning the charge as described.
2. In the main program, use a loop to collect each vehicle's plate and hours.
3. For each vehicle, call the function to work out the fee.
4. Display the result in a clean, formatted output.

(e) Develop a new car park till as an extension of (d). This version should handle any number of vehicles in one session. **[3.5mks]**

Program requirements:

1. Define `fee(hours)` as before.
2. Use a loop to repeatedly accept a plate number and hours.
3. Stop the loop when the user enters 'done' as the plate number.
4. For each vehicle, call the function and display the fee.
5. After the loop, display how many vehicles were charged and the total takings.

Question Four

QUESTION FOUR

17.5 MARKS

(a) What is a string in Python, and what does it mean to say a string is immutable? What follows from that when you call a method on one? List four string methods with an example of each, **at least one of which asks a question about the string rather than changing it.** [3.5mks]

(b) The following code is intended to print the domain of an email address in lower case. It contains two errors, and the second is hidden behind the first. Identify and fix them: [3.5mks]

```
email = "Ada@CU.edu.ng"
domain = email.split("@")
print("Domain:", domain.lower)
```

(c) What will be the output of the following code? Explain the result of each line: [3.5mks]

```
code = "  csc241-python  "
print(code.strip().upper())
print(code.strip().split("-"))
print(len(code.strip()))
```

(d) Write a Python program for a tool that processes a list of staff email addresses and prints the domain of each. An address has the form `name@domain`, and the domain is the part after the `@`. [3.5mks]

Program Requirements:

1. Create a list containing at least five email addresses.
2. Loop through the list.
3. For each one, split the string at the `@` and take the domain.
4. Print the result in a clean format, such as:

```
Email: ada@cu.edu.ng | Domain: cu.edu.ng
```

(e) Develop a new text tool as an extension of (d). This version should check a batch of product codes typed in a single line. [3.5mks]

Program Requirements:

1. Prompt the user to enter several product codes separated by commas.
2. Split the line at the comma.
3. For each code: remove leading and trailing whitespace, skip it if it is empty, and put it in capitals.
4. For each code, display how many of its characters are letters, how many are not letters, and whether it begins with "CSC".

Question Five

QUESTION FIVE

17.5 MARKS

(a) What does `with open(...)` give you that a plain `open()` does not? Explain the modes `"r"`, `"w"` and `"a"`, and say what happens in each if the file does not yet exist. **[3.5mks]**

(b) The following code is intended to add a name to a file. It contains two errors. Identify and fix them: **[3.5mks]**

```
f = open("names.txt", "r")
f.write("Ada\n")
f.close
```

(c) What will the following code print? Explain what happens to the first line that was written, and why. **[3.5mks]**

```
with open("data.txt", "w") as f:
    f.write("first\n")

with open("data.txt", "w") as f:
    f.write("second\n")

with open("data.txt") as f:
    print(f.read())
```

(d) Develop a Python program for a lecturer's attendance sheet. The program should allow the lecturer to enter the names of the students present today and save the list to a file called `attendance.txt`. **[3.5mks]**

Program Requirement:

1. Ask the lecturer how many students are present.
2. Use a loop to input each name.
3. Open `attendance.txt` in write mode.
4. Write each name to the file on a new line.
5. After saving, print "Attendance saved successfully."

(e) Develop a new results recorder as an extension of (d). This version should allow the user to enter multiple records (name and score) until 'done' is entered as the name. Each entry should be stored in a text file called `scores.txt` in the format: **[3.5mks]**

```
Ada, 72
Bola, 45
```

Program Requirement:

1. Open `scores.txt` in write mode.
2. Use a loop to collect names and scores until 'done' is entered.
3. Write each entry in the format: Name, Score.
4. Close the file, then reopen it in read mode.
5. For each line: split the name and the score, print the record, and keep a running total. **A pass is 50 and above.**
6. Display the class average and how many students passed.

Question Six

QUESTION SIX

17.5 MARKS

(a) i. In a database, what is a table, a row and a column? ii. What does `conn.commit()` do, and what happens if you forget it? iii. What is the difference between `fetchone()` and `fetchall()`? **[3.5mks]**

(b) The following code is meant to create a `products` table and put one row in it, but it contains errors. Identify and fix the issues: **[3.5mks]**

```
import sqlite3
conn = sqlite3.connect("shop.db")
cursor = conn.cursor()
fields = "(id INTEGER PRIMARY KEY, name TEXT, price INTEGER)"
cursor.execute("CREATE TABLE products " + fields)
cursor.execute("INSERT INTO products (name, price) VALUE ('Pen', 150)")
conn.commit()
conn.close
```

(c) i. Examine the following code and explain what it does. ii. What will it print? iii. What assumptions must be true for it to execute successfully? **[3.5mks]**

```
import sqlite3
conn = sqlite3.connect("shop.db")
cursor = conn.cursor()
cursor.execute("SELECT name FROM products")
print(cursor.fetchall())
conn.close()
```

(d) Develop a simple Python-based shop inventory using `sqlite3`. The program should create a table `products` with fields: `id`, `name` and `price`. It prompts the user to enter the name and price of three products and inserts each into the database. **[3.5mks]**

Program requirements:

1. Import `sqlite3` and connect to a database named `inventory.db`.
2. Create a table `products` if it does not already exist.
3. Use a loop to collect the name and price of three products.
4. Insert each product into the table.
5. Commit the changes and close the connection.

(e) Develop a new inventory as an extension of (d). This version should allow the user to enter products until they type 'done' as the name, store the records in `inventory.db`, and after all entries are added, fetch and display every product stored in the table. **[3.5mks]**

The answers follow. **Do not read them until you have written yours.**

You have seen one set of answers already, and that makes this page more dangerous, not less: the shapes are familiar now, and familiarity reads exactly like competence until you are asked to produce one from nothing. If you can write Q1(e) without looking, you have learned the pattern. If you cannot, you have only recognised it, and recognition earns no marks.

Answers

Every output below was produced by running the code, not by reading it. Where your answer differs from the marking, the difference is information: find out which of you is right before you move on.

Question One

ANSWER

Q1 (A)

A **control structure** decides which statements run, and how often. Without one a program is a list that runs top to bottom, once.

- ▶ **break** leaves the loop immediately. Nothing after it in the loop body runs, and the loop does not continue. `for i in range(5): if i == 2: break`
- ▶ **continue** abandons *this pass only* and jumps to the next. The loop carries on. `for i in range(5): if i % 2: continue`
- ▶ **else on a loop** runs when the loop finishes **without being broken**. It is the part everyone forgets: it is not "otherwise", it is "and we got all the way through".

The three-way answer the examiner wants: sequence, selection (`if`), repetition (`for` , `while`). `break` and `continue` steer the third.

ANSWER

Q1 (B)

Two errors, and the second only appears once the first is gone.

- 1 The `for` line has **no colon**. Python stops before running anything:
SyntaxError: expected ':'
- 2 Add the colon and it runs, and now fails on the last line:
NameError: name 'Total' is not defined
`Total` is not `total` . Python is case sensitive, and a name with a capital letter is a different name.

FIXED

```
total = 0
for i in range(1, 5):
    total += i
print("Total:", total)
```

Total: 10

`range(1, 5)` gives 1, 2, 3, 4. The 5 is the stop and never arrives, so the total is 10 and not 15. If you answered 15 you have the off by one that this part exists to catch.

ANSWER

Q1 (C)

```
3
2
1
Liftoff
```

`range(3, 0, -1)` counts **down**: start at 3, stop before 0, step by -1. So 3, 2, 1, and never 0.

Why the last line prints at all is the whole question. A loop's `else` runs when the loop **completes without a break**. There is no `break` here, so the loop finishes, and the `else` fires.

This is the mirror of Mock One Q1(c), where a `break` did fire and the `else` was skipped. Same rule, both sides of it. If you can state that rule in one sentence you have both marks.

ANSWER

Q1 (D)

BILL.PY

```
units = int(input("Enter units consumed: "))

if units > 0:
    if units < 50:
        band, rate = "Lifeline", 10
    elif units < 200:
        band, rate = "Domestic", 20
    else:
        band, rate = "Commercial", 30
    print(f"Band: {band} | Bill: N{units * rate}")
else:
    print("Please enter a valid reading")
```

Entering 120:

```
Band: Domestic | Bill: N2400
```

The `elif` chain is doing work you get for free: by the time Python reaches `elif units < 200` it already knows the units are **not** under 50, so you never write `units >= 50` and `units < 200`. Writing that instead is not wrong, but it is the long way round and it is where people put the boundary in the wrong place.

The boundaries decide the mark:

UNITS	BAND	BILL
49	Lifeline	N490
50	Domestic	N1000
199	Domestic	N3980
200	Commercial	N6000

50 is **not** under 50, so it is Domestic. 200 is not under 200, so it is Commercial. Read `<` as "strictly less than" every time, and these four fall out on their own.

Same banding, wrapped in **split and convert**. Nothing about the pricing changes. That is the point of "as an extension of (d)": do not rewrite it, wrap it.

BILLS.PY

```
line = input("Enter the readings separated by spaces: ")

count = 0
grand = 0
for part in line.split():
    units = int(part)
    if units > 0:
        if units < 50:
            band, rate = "Lifeline", 10
        elif units < 200:
            band, rate = "Domestic", 20
        else:
            band, rate = "Commercial", 30
        bill = units * rate
        grand += bill
        count += 1
        print(f"{units} units | {band} | N{bill}")
    else:
        print(units, "Please enter a valid reading")

print("Readings billed:", count)
print("Total:", grand)
```

Entering 30 120 250 :

```
30 units | Lifeline | N300
120 units | Domestic | N2400
250 units | Commercial | N7500
Readings billed: 3
Total: 10200
```

Three things earn the marks: `split()` with no argument (any run of spaces), `int(part)` on every piece because `input()` hands you text, and `count` incremented **inside** the valid branch, so a rejected reading is not counted as billed.

Question Two

ANSWER

Q2 (A)

STRUCTURE	WHAT IT IS	USE IT FOR
List []	Ordered, changeable, duplicates allowed	The parcels weighed today: the set of them changes all day.
Tuple ()	Ordered, unchangeable	A fixed pair like (latitude, longitude), or the days of the week.
Set {}	Unordered, no duplicates	The distinct courses a student takes: ask it "how many different?"
Dictionary {k: v}	Keyed pairs	Stock levels: look up "pen" directly, without searching.

List against tuple: use a tuple when the collection must not change, and the fact that it cannot is the protection. A list is the default; reach for a tuple deliberately.

What a dictionary gives that a list cannot: it finds a value **by name**. In a list you must know the position, or search the whole thing. In a dictionary `stock["pen"]` goes straight there, however many items there are.

ANSWER

Q2 (B)

Two errors, the second hidden behind the first.

- 1 `weights` is a **tuple**, and a tuple cannot be changed:
AttributeError: 'tuple' object has no attribute 'append'
The collection is being added to, so it should have been a list: `[12, 5, 20, 8]`.
- 2 Make it a list and the crash stops, but the output is wrong:
Sorted: None
`sort()` sorts the list **in place** and returns `None`. You printed its return value, which is nothing. Sort first, then print the list.

FIXED

```
weights = [12, 5, 20, 8]
weights.append(15)
weights.sort()
print("Sorted:", weights)
```

```
Sorted: [5, 8, 12, 15, 20]
```

The second error is the one worth remembering: **a method that changes a thing in place gives back None**. `sort()`, `reverse()` and `append()` are all like this. If you want a sorted copy and the original kept, that is `sorted(weights)`, which returns a new list.

ANSWER

Q2 (C)

```
9
3
True
```

- 1 `stock["pen"] = stock["pen"] - 3` reads 12, takes 3, and writes 9 back to the **existing** key. The dictionary still has two keys.
- 2 `stock["bag"] = 7` uses a key that is not there, so it is **created**. The same square brackets both update and add: which one happens depends entirely on whether the key already exists.
- 3 `print(stock["pen"])` gives **9**.
- 4 `len(stock)` counts the **keys**: pen, book, bag, so **3**. Not the values, and not the characters.
- 5 `"book" in stock` asks whether that is a **key**, and it is: **True**. On a dictionary, `in` looks at the keys, never the values. `12 in stock` would be False even though 12 is in there.

ANSWER

Q2 (D)

PARCELS.PY

```
weights = []

for i in range(4):
    w = int(input(f"Weight of parcel {i + 1}: "))
    weights.append(w)

weights.sort()
print("Sorted:", weights)
print("Heaviest:", max(weights))
print("Lightest:", min(weights))
```

Entering 12, 5, 20, 8:

```
Sorted: [5, 8, 12, 20]
Heaviest: 20
Lightest: 5
```

"Appropriate built-in functions" means `max()` and `min()`. You could take `weights[-1]` and `weights[0]` after sorting and be right, but only because you sorted first; `max()` needs no such promise and says what it means.

PARCELS_ANY.PY

```
line = input("Enter all the weights separated by spaces: ")

weights = []
for part in line.split():
    weights.append(int(part))

weights.sort(reverse=True)
average = sum(weights) / len(weights)

surcharge = 0
for w in weights:
    if w > 20:
        surcharge += 1

print("Sorted:", weights)
print("Heaviest:", max(weights))
print("Lightest:", min(weights))
print(f"Average: {average:.2f}")
print("Surcharged:", surcharge)
```

Entering 12 5 20 8 25 :

```
Sorted: [25, 20, 12, 8, 5]
Heaviest: 25
Lightest: 5
Average: 14.00
Surcharged: 1
```

- ▶ **Descending** is `sort(reverse=True)` . It still returns `None` , so it is still a statement on its own line.
- ▶ **Only one parcel is surcharged.** The rule is *above* 20, so the 20 kg parcel does not qualify: 25 does. If you answered 2 you read `>` as `>=` , which is the single most expensive habit in this paper.
- ▶ The average is 70 divided by 5, which is 14.0 exactly. `{average:.2f}` shows it as 14.00: two decimal places for money and measurements, always.

Question Three

ANSWER

Q3 (A)

- ▶ **Parameter against argument.** The **parameter** is the name in the definition, `def fee(hours)`. The **argument** is the actual value you pass, `fee(3)`. One is the label, the other is the thing.
- ▶ **return** hands a value back to whoever called the function, and ends the function there and then. Leave it out and the function still returns something: it returns **None**. That is not an error, which is exactly why it is dangerous, and it is what part (b) is about.
- ▶ **A default parameter** supplies a value for an argument the caller does not give: `def pay(hours, rate=500)`. It **must come last** because Python matches arguments to parameters by position. If a default came first, then `pay(8)` could not say whether the 8 is the defaulted one or the required one. Python refuses the definition outright rather than guess.

ANSWER

Q3 (B)

Two errors, and the second is invisible until the first is fixed.

- 1 `def fee(hours)` has **no colon**. Nothing runs at all:
SyntaxError: expected ':'
- 2 Add it and the program runs cleanly and prints the wrong thing:
None
The function works out `total` and then **throws it away**. There is no `return`, so the function gives back `None`, and `print` faithfully prints it. No error, no warning, no marks.

FIXED

```
def fee(hours):  
    return 200 + (hours - 1) * 150  
  
print(fee(3))
```

500

200 for the first hour, then 2 more hours at 150, so $200 + 300 = 500$. **A function that computes and does not return is the most common quiet failure in this course.** If your function prints `None`, this is why, every time.

ANSWER

Q3 (C)

```
4000
5600
```

- ▶ `pay(8)` gives one argument, so `hours` is 8 and `rate` falls back to its default of 500: $8 * 500 = \mathbf{4000}$.
- ▶ `pay(8, 700)` gives two, so the second **replaces** the default: $8 * 700 = \mathbf{5600}$.

The default is not a constant. It is the value used *only* when the caller stays silent. That is what makes `rate` optional without making it fixed, and it is the whole reason default parameters exist.

ANSWER

Q3 (D)

CARPARK.PY

```
def fee(hours):
    return 200 + (hours - 1) * 150

for i in range(3):
    plate = input("Plate number: ")
    hours = int(input("Hours parked: "))
    print(f"Plate: {plate} | Hours: {hours} | Fee: N{fee(hours)}")
```

Entering ABC123/3, XYZ789/1 and KJA55/5:

```
Plate: ABC123 | Hours: 3 | Fee: N500
Plate: XYZ789 | Hours: 1 | Fee: N200
Plate: KJA55 | Hours: 5 | Fee: N800
```

Check the one-hour case by hand, because it is the one the formula can get wrong: $200 + (1 - 1) * 150 = 200 + 0 = 200$. Correct. Write the formula as `200 + hours * 150` and that vehicle is charged 350 for its first hour, and every single answer is 150 too high.

The **sentinel loop**: while True, ask, and break on the stop word. The function itself does not change at all.

CARPARK_ANY.PY

```
def fee(hours):  
    return 200 + (hours - 1) * 150  
  
count = 0  
takings = 0  
  
while True:  
    plate = input("Plate number (or 'done'): ")  
    if plate == "done":  
        break  
    hours = int(input("Hours parked: "))  
    amount = fee(hours)  
    takings += amount  
    count += 1  
    print(f"Plate: {plate} | Hours: {hours} | Fee: N{amount}")  
  
print("Vehicles charged:", count)  
print(f"Total takings: N{takings}")
```

Entering ABC123/3, XYZ789/1, then done :

```
Plate: ABC123 | Hours: 3 | Fee: N500  
Plate: XYZ789 | Hours: 1 | Fee: N200  
Vehicles charged: 2  
Total takings: N700
```

Test the stop word before you ask for the hours. Put the `int(input(...))` above the check and typing `done` makes the program demand the hours for a vehicle that does not exist, and then crash on `int("")`. The check goes immediately after the input it checks.

Question Four

ANSWER

Q4 (A)

A **string** is text: a sequence of characters in quotes. It is **immutable**, meaning it can never be changed after it is made.

What follows from that is the examinable half: a string method can never change the string it is called on, so it hands you a **new** one. `name.upper()` leaves `name` exactly as it was. If you do not catch the result, with `name = name.upper()`, the work is thrown away and nothing happens. This is the opposite of a list's `sort()`, which changes the list and hands back nothing.

METHOD	DOES	EXAMPLE	GIVES
<code>upper()</code>	Returns a capitalised copy	<code>"ada".upper()</code>	<code>'ADA'</code>
<code>strip()</code>	Returns a copy with the ends trimmed	<code>" hi ".strip()</code>	<code>'hi'</code>
<code>split(sep)</code>	Cuts it into a list	<code>"a,b".split(",")</code>	<code>['a', 'b']</code>
<code>isalpha()</code>	Asks a question, changes nothing	<code>"Ada".isalpha()</code>	<code>True</code>

The fourth is the one the question is fishing for. `isalpha()`, `startswith()` and `find()` interrogate a string rather than transform it; the rest hand back a changed copy.

ANSWER

Q4 (B)

Two errors, the second wearing the first as a disguise.

- 1 `split("@")` gives a **list**, `['Ada', 'CU.edu.ng']`, not a string. So `domain` is a list, and a list has no `lower`:

AttributeError: 'list' object has no attribute 'lower'

You want the piece after the `@`, which is index **1**: `email.split("@")[1]`.

- 2 Take the index and the crash goes away, and the output is still wrong:

Domain: <built-in method lower of str object at 0x...>

`domain.lower` has **no brackets**, so it never calls the method. It names it. Python prints the method itself, which is a real object and prints happily. (The `0x...` is a memory address, and it is a different number every time the program runs.)

FIXED

```
email = "Ada@CU.edu.ng"
domain = email.split("@")[1]
print("Domain:", domain.lower())
```

```
Domain: cu.edu.ng
```

The missing brackets again. This is the third time this manual has shown you the same fault: `file.close`, `conn.cursor`, and now `domain.lower`. **A method without brackets is not called, and Python never complains.** When output looks like `<built-in method ...>` or `<bound method ...>`, you have found it.

ANSWER

Q4 (C)

```
CSC241-PYTHON
['csc241', 'python']
13
```

- 1 `code.strip()` gives `'csc241-python'`, and `.upper()` capitalises that copy. The chain reads left to right: strip first, then capitalise the result.
- 2 `split("-")` cuts the stripped string at the hyphen into **two** pieces. They stay lowercase, because `upper()` on the line above returned a new string and did not touch `code`.
- 3 `len(code.strip())` is **13**: `c-s-c-2-4-1` is 6, the hyphen is 1, `p-y-t-h-o-n` is 6. `len(code)` without the strip would be **17**, counting the four spaces.

`code` itself is unchanged throughout, spaces and all. Three methods were called on it and not one of them altered it: that is immutability doing exactly what part (a) says it does.

ANSWER

Q4 (D)

DOMAINS.PY

```
emails = ["ada@cu.edu.ng", "bola@gmail.com", "chidi@yahoo.com",
          "dele@cu.edu.ng", "efe@outlook.com"]

for email in emails:
    domain = email.split("@")[1]
    print(f>Email: {email} | Domain: {domain}")
```

```
Email: ada@cu.edu.ng | Domain: cu.edu.ng
Email: bola@gmail.com | Domain: gmail.com
Email: chidi@yahoo.com | Domain: yahoo.com
Email: dele@cu.edu.ng | Domain: cu.edu.ng
Email: efe@outlook.com | Domain: outlook.com
```

Index **1**, not 0. Splitting `"ada@cu.edu.ng"` at the `@` gives `['ada', 'cu.edu.ng']`: the name is 0 and the domain is 1. The dots in `cu.edu.ng` are harmless, because we split on `@` and the dots are all inside a piece we never cut again.

CODES.PY

```
line = input("Enter the codes separated by commas: ")

for code in line.split(","):
    code = code.strip()
    if code == "":
        continue
    code = code.upper()

    letters = 0
    for ch in code:
        if ch.isalpha():
            letters += 1

    others = len(code) - letters
    csc = code.startswith("CSC")
    print(f"Code: {code} | Letters: {letters} | Not letters: {others} | CSC: {csc}")
```

Entering csc241, mat121x, , abc:

```
Code: CSC241 | Letters: 3 | Not letters: 3 | CSC: True
Code: MAT121X | Letters: 4 | Not letters: 3 | CSC: False
Code: ABC | Letters: 3 | Not letters: 0 | CSC: False
```

- ▶ **Four pieces went in and three came out.** The empty one between the two commas is skipped by the `continue`, exactly as the requirement asks. This is the same guard as Mock One's paragraph, and it exists because real input has gaps in it.
- ▶ **Capitalise before you test.** `startswith("CSC")` is case sensitive, so `"csc241"` would answer False if you tested it before `upper()`. Order matters, and the requirement lists it in the right order for a reason.
- ▶ **Not letters is `len(code) - letters`**, not a second loop. You have already counted the letters, and everything else is what remains.

Question Five

ANSWER

Q5 (A)

`with open(...)` closes the file for you, whatever happens, even if the code inside raises an error. A plain `open()` leaves that to you, and if an exception fires before your `close()`, the file is never closed and the last thing you wrote may never reach the disk. `with` is not a convenience; it is the guarantee.

MODE	DOES	IF THE FILE DOES NOT EXIST
"r"	Read only. Cannot write.	Error. It refuses to create it.
"w"	Write, and empty the file first.	Creates it.
"a"	Append: add to the end, keep what is there.	Creates it.

Reading against appending: "r" takes data out and changes nothing; "a" puts data in at the end and destroys nothing. The dangerous one is "w", which silently throws away the whole file the instant you open it.

ANSWER

Q5 (B)

Two errors.

- 1 "r" is **read only**, and the program is trying to write. The mode should be "a" to add a name and keep the others.
- 2 `f.close` has **no brackets**, so the file is never closed. Same fault as Q4(b), in a different coat.

The first error gives two different messages, and which one you get depends on something the code does not show you:

IF NAMES.TXT	PYTHON SAYS
exists	UnsupportedOperation: not writable
does not exist	FileNotFoundError: [Errno 2] No such file or directory: 'names.txt'

With the file missing, `open()` itself fails and `write` is never reached, so you never hear about writability at all. Both answers are correct, and saying **why there are two** is worth more than either.

FIXED

```
f = open("names.txt", "a")
f.write("Ada\n")
f.close()
```

Better still, let `with` do the closing and the second error cannot happen at all:

```
with open("names.txt", "a") as f:
    f.write("Ada\n")
```

ANSWER

Q5 (C)

```
second
```

Followed by a blank line, because the file's contents end in `\n` and `print` adds another.

The first line is gone. Opening in `"w"` empties the file immediately, before a single byte is written. The second `with` block therefore starts from nothing, and `"first"` is destroyed without a word.

Had the second block used `"a"`, the file would hold both lines. This one question is the whole difference between the two modes, and it is why a program that saves records in a loop must open the file **once**, outside the loop: reopen it in `"w"` each time round and you finish with a file containing only the last record.

ANSWER

Q5 (D)

ATTENDANCE.PY

```
count = int(input("How many students are present? "))

with open("attendance.txt", "w") as f:
    for i in range(count):
        name = input(f"Name of student {i + 1}: ")
        f.write(name + "\n")

print("Attendance saved successfully.")
```

Entering 3, then Ada, Bola, Chidi:

```
Attendance saved successfully.
```

and `attendance.txt` holds:

```
Ada
Bola
Chidi
```

`write()` adds **no** newline of its own, unlike `print`. Leave off the `+ "\n"` and the whole register lands on one line as `AdaBolaChidi`. The file is opened once, outside the loop, for the reason part (c) just demonstrated.

SCORES.PY

```
f = open("scores.txt", "w")

while True:
    name = input("Name (or 'done'): ")
    if name == "done":
        break
    score = input("Score: ")
    f.write(name + ", " + score + "\n")

f.close()

f = open("scores.txt", "r")
total = 0
count = 0
passes = 0

for line in f:
    name, raw = line.strip().split(", ")
    score = int(raw)
    total += score
    count += 1
    if score >= 50:
        passes += 1
    print(f"{name}: {score}")

f.close()
print(f"Average: {total / count:.2f}")
print("Passed:", passes)
```

Entering Ada/72, Bola/45, Chidi/58, then done :

```
Ada: 72
Bola: 45
Chidi: 58
Average: 58.33
Passed: 2
```

- ▶ **Bola fails.** A pass is 50 **and above**, and 45 is not. Two of the three passed. The average is $175 / 3 = 58.33$, which is not a pass mark for anybody: it is just the class average.
- ▶ `line.strip()` before `split(", ")`, or the newline sticks to the score and `int("72\n")` ... in fact survives, because `int()` ignores surrounding whitespace. Strip anyway: relying on that is a habit that breaks the moment you compare the text instead of converting it.
- ▶ **Split on ", " including the space**, because that is exactly how you wrote it. Split on `","` and the score arrives as `" 72"`. The file format and the parser are one decision, made twice.

Question Six

ANSWER

Q6 (A)

i. A **table** is one kind of thing being stored, like products. A **row** is one of them: one product. A **column** is one fact held about every one of them: the price. A table is a grid, rows down, columns across.

ii. `conn.commit()` makes your changes **permanent**. Until you call it, inserts live in an open transaction and belong only to your connection. Forget it and the program runs, prints nothing wrong, exits, and **the database is unchanged**. No error is ever raised. It is the single most common way to lose work in this topic.

iii. `fetchone()` returns **one** row and moves on, or `None` if there are none left. `fetchall()` returns **every** remaining row, as a list of tuples, and an empty list if there are none. Use `fetchone()` when you expect a single answer, `fetchall()` when you want to loop over the lot.

ANSWER

Q6 (B)

Two errors.

- 1 `VALUE` should be **VALUES**. This is SQL, not Python, so Python cannot help you: the string is perfectly good Python and the database is the one that objects.
OperationalError: near "VALUE": syntax error
- 2 `conn.close` has **no brackets**, so the connection is never closed. The third appearance of this fault in two mocks. Note that the program never even reaches it here: the error above stops everything first.

FIXED

```
import sqlite3

conn = sqlite3.connect("shop.db")
cursor = conn.cursor()
fields = "(id INTEGER PRIMARY KEY, name TEXT, price INTEGER)"
cursor.execute("CREATE TABLE IF NOT EXISTS products " + fields)
cursor.execute("INSERT INTO products (name, price) VALUES ('Pen', 150)")
conn.commit()
conn.close()
```

`IF NOT EXISTS` is added for a reason worth a mark on its own: run the original twice and the second run dies with `OperationalError: table products already exists`. A program that only works once is not finished.

i. It opens `shop.db`, asks for the `name` column of every row in `products`, prints all the rows it gets back, and closes the connection.

ii. With the single `Pen` row from (b) in the table:

```
[('Pen',)]
```

A **list of tuples**, one tuple per row, even though only one column was asked for. That trailing comma in `('Pen',)` is Python's way of writing a tuple of one item, and it is not a typo. To print plain names you would loop and take `row[0]`.

iii. Three assumptions:

1. `shop.db` can be opened. `connect()` **creates** the file if it is absent, so this never fails, and that is the trap.
2. A table called `products` exists in it, with a `name` column. If not: `OperationalError: no such table: products` which is exactly what you get if `connect()` just made you a fresh, empty database because you misspelled the filename.
3. The rows were **committed** by whoever inserted them. Uncommitted inserts from another connection are invisible here.

INVENTORY.PY

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()
fields = "(id INTEGER PRIMARY KEY, name TEXT, price INTEGER)"
cursor.execute("CREATE TABLE IF NOT EXISTS products " + fields)

for i in range(3):
    name = input("Product name: ")
    price = int(input("Price: "))
    cursor.execute("INSERT INTO products (name, price) VALUES (?, ?)", (name, price))

conn.commit()
conn.close()
print("Three products saved.")
```

Three products saved.

- ▶ **The ? placeholders.** Pass the values as a tuple second and let `sqlite3` put them in. Build the SQL by gluing strings together and a product named `O'Brien` ends your statement early and crashes it.
- ▶ **id is not inserted.** `INTEGER PRIMARY KEY` fills itself in: 1, 2, 3.
- ▶ **`conn.commit()` is not optional.** Leave it out and this program prints "Three products saved." and saves nothing at all.

The sentinel loop from Q3(e), with the database from (d) behind it.

INVENTORY_ANY.PY

```
import sqlite3

conn = sqlite3.connect("inventory.db")
cursor = conn.cursor()
fields = "(id INTEGER PRIMARY KEY, name TEXT, price INTEGER)"
cursor.execute("CREATE TABLE IF NOT EXISTS products " + fields)

while True:
    name = input("Product name (or 'done'): ")
    if name == "done":
        break
    price = int(input("Price: "))
    cursor.execute("INSERT INTO products (name, price) VALUES (?, ?)", (name, price))

conn.commit()

cursor.execute("SELECT * FROM products")
for row in cursor.fetchall():
    print(row)

conn.close()
```

Entering Pen/150, Book/1200, then done :

```
(1, 'Pen', 150)
(2, 'Book', 1200)
```

Commit before you select. The order is: loop, commit, then read back. And `close()` comes last of all, once: put it inside the loop and the second product is inserted through a closed connection.

`SELECT *` gives every column, so each row arrives as the full tuple including the id the database made for you. That is why these tuples have three items and Q6(c)'s had one.

Two papers, sixty parts, and underneath them **four** patterns. Learn these and you have not learned two mocks, you have learned the exam.

PART	ALWAYS ONE OF
(b)	A first error that hides a second. Fix the crash and read the output again: it is usually None, or a method printed instead of called.
(c)	A rule stated from one side. Mock One broke the loop so else was skipped; Mock Two did not, so it ran.
(d)	A banding decided by boundaries. 49 against 50, 199 against 200, "under 13" against "13 and above".
(e)	Sentinel loop (while True, break on 'done') or split and convert (one line, split(), int() each piece). Nothing else has ever been set.

The one habit worth more than any of them: read `<` as "strictly less than" and `>` as "strictly more than", out loud, every single time. A parcel above 20 kg is not a parcel of 20 kg. A pass of 50 and above includes 50. Every (d) and every (e) in both papers turns on that, and it is the cheapest mark on the paper to lose.